MacroBase

Bug and reliability audit report

Project	MacroBase, an offline-first Android nutrition tracker (version 1.2.7, versionCode 18)
Date of audit	31 October 2026
Version control	Branch `master`, latest commit `32bf88f`, plus the uncommitted changes in the working tree at audit time
Scope audited	The whole codebase: the Android app, its tests, the catalog tools, the two bundled databases and the documentation
Findings	67 in total: 0 P0, 1 P1, 16 P2, 36 P3, 14 P4

This was a read-only static review. No code was executed or modified.

Contents

B. Executive summary

C. Scope, method and coverage

D. Priority table

E. Detailed findings

F. User journey failures

G. Root-cause patterns

H. Recommended fix order

I. Test plan

J. Remaining uncertainty

B. Executive summary

MacroBase is in better shape than the size of this list suggests. The audit found **67 issues**, with **no P0** and **one P1**. The foundations held up under deliberate attempts to break them: Room migrations and the no-destructive-fallback rule, diary snapshots at log time, the basket's commit guards, custom-food scaling, null handling for nutrients, the backup archive parser, the rank's season maths, and the zero-network build guard. No security vulnerability of note was found, and no secrets.

The problems sit around those foundations: the label scanner's parser, state that is lost or goes stale across screen lifecycle events, and a few paths that an earlier fix did not reach.

Priority	Meaning	Total	Confirmed	Highly Likely	Potential
P0	Critical	0	0	0	0
P1	High	1	1	0	0
P2	Medium	16	14	2	0
P3	Low	36	24	10	2
P4	Informational	14	14	0	0
All		**67**	**53**	**12**	**2**

Most dangerous issues

- BUG-001 (P1): the label scanner gives every nutrient on a shared line the same number, so a side-by-side or paragraph-style label fills in wrong macros, including Protein 0 g, with a green tick and no warning.
- BUG-010 (P2): pressing Back during a restore commits the database but skips goals and profile, with no message at all.
- BUG-011 (P2): a Merge onto a new phone drops the backup's goal history, so imported days leave the rank and past days are judged against goals they never had.
- BUG-008 (P2): changing the amount on an old entry recalculates it from the food's current values, which breaks the immutable-history rule in AGENTS.md section 1.4.
- BUG-002 (P2): rotating the phone or visiting the scanner wipes a half-built recipe or custom food.
- BUG-003 and BUG-006 (P2): food, water and weight can land on the wrong day, and a morning weigh-in can overwrite yesterday's entry.
- BUG-007 (P2): saving a new weight can silently switch the user's strategy, which then costs rank points on good days.

User journeys most at risk

- **Scanning a nutrition label.** The P1, six of the sixteen P2 findings and eleven P3 findings are on this path. Review is always shown before saving, but the green ticks imply a check that did not happen.
- **Logging for any day other than today.** Several entry points ignore the day on screen.
- **Moving to a new phone.** Restore has two serious gaps, both silent.
- **Finding a food.** Plural searches such as "bananas" return nothing (BUG-004).

Overall assessment of production readiness

Ready for continued use as a daily tracker, with the scanner treated as beta. Nothing here endangers the stored diary: no finding deletes or corrupts existing diary rows outside the user's own actions, and the data layer is strong. Before a wider release, fix BUG-001, BUG-010, BUG-011, BUG-008 and BUG-002, then the day-handling group. Until the parser findings are fixed, removing the green ticks would be a cheap and honest stopgap.

The test suite has about 840 unit tests and is strong on logic, but it has no Compose UI tests, and its scanner fixtures hide real ML Kit behaviour. Most findings here fall in those two gaps.

C. Scope, method and coverage

Type of system

An Android mobile app with no server and no network access, plus a Python data pipeline that builds its two bundled SQLite catalogs. There is no authentication, payment or backend, so those parts of the audit brief do not apply.

- **App:** Kotlin 2.1.10, Jetpack Compose (BOM 2025.02.00), Navigation Compose, Room 2.6.1, DataStore 1.1.3, Koin 4.0.2, coroutines 1.10.1, CameraX 1.4.1, ML Kit text recognition 16.0.1 (bundled model). minSdk 26, targetSdk 35.
- **Pipeline:** Python scripts and CSV curation files in `tools/catalog`.
- **Size:** 146 Kotlin source files (about 34,500 lines), 72 test files (about 23,500 lines, about 840 tests), 18 documents, 7 Python files, 2 catalogs (977 and 1,632 foods).

Method

The audit followed the requested phases: reconnaissance, a risk map, a deep review in risk order, cross-cutting checks, verification, then this report. Six read-only auditors each took one area, under the same read-only rules: backup, scanner, goals and rank, logging screens, trackers and shell, and catalog data. The lead auditor reviewed the data layer, migrations, domain models, basket pipeline and cross-cutting rules directly.

Verification. The lead auditor re-opened the code behind every finding and traced it. Scanner findings were hand-traced against sample label text, since the parser could not be run. Some priorities were changed in that step, mostly downward, and overlapping reports were merged. Each code quote in this report is copied from the file when the report is built, and the build stops if the expected text is not on the cited lines.

Security review

No security finding reached P3 or above. Checked with nothing material found: exported components (the widget receiver only redraws), SQL built from strings (queries are parameterised), zip path traversal and zip bombs (nothing is extracted to disk, and limits hold), the manifest's INTERNET removal and its build guard, cloud backup exclusions, and secrets in code, config and history. `gradle.properties` holds a certificate fingerprint, which is public, not a secret. The only privacy issue is BUG-060. Unencrypted local storage and unencrypted backup archives are documented, accepted limitations. No file in the codebase attempted to give instructions to the auditors.

Coverage

Level	Areas
Reviewed fully	Room database, migrations, DAOs, entities; catalog manager and provider; diary, food, recipe, water, weight, statistics, goals, rank, portability and preferences repositories; basket pipeline; domain models and use cases; backup archive and JSON; snapshot cache; all of `feature/scanner`; Home, Search, Food Detail, Basket, Custom Foods, Recipes, Water, Weight, Calendar, Statistics, Rank and Import screens; navigation; Application, MainActivity, DI; widget; manifest, Gradle build, R8 rules, backup rules; all Python tools and their tests; both databases, by query.
Reviewed partly	Preferences screen (lines 140-915); design-system components (behaviour, not visual details); roughly 45 of the 72 test files, read for what they assert; `corrections.csv`, `exclusions.csv` and `common_foods.csv`, by query and sampling; the root foods CSV, by header and sampled rows; documents that are historical reports.
Not reviewed	`ImageQualityCheckerUnitTests.kt`; the instrumented test; design tokens (colours, spacing, shapes) and previews; the source files behind the catalogs (INDB workbook, USDA and CoFID downloads, Open Food Facts snapshot), which are not in the repository.

Level	Areas
Skipped by rule	Generated and third-party material: `.gradle`, `.kotlin`, `.idea`, `app/build`, the Gradle wrapper, vector drawables, fonts, and `rank-img`.

About 250 files were reviewed in all.

What was run

The project was not built, run or tested. These read-only steps were used:

- file reads and searches, and read-only git commands;
- queries on the two bundled databases through Python's `sqlite3` with `mode=ro&immutable=1`;
- a small Python port of the search SQL, run against those files in memory, for the ranking results in BUG-004 and BUG-046;
- `javap` and `unzip -l` on the cached room-ktx library, for BUG-010.

One side effect occurred outside the project: while checking for a PDF tool, running `brave.exe --version` started the browser, which printed "Opening in existing browser session" and may have opened a window. It changed nothing in the repository.

D. Priority table

Sorted from highest priority to lowest. Each ID links to its finding.

ID	Priority	Issue	User impact	Confidence	Location
BUG-001	P1	Label scanner gives every nutrient on a shared line the same number	Side-by-side and paragraph-style labels fill in wrong macros, including Protein 0 g, marked with a green tick.	Confirmed	NutritionLabelParser.kt:1119-1275
BUG-002	P2	A half-filled custom food or recipe is wiped by rotation, a trip to the scanner, or Back	Typed food details, whole ingredient lists and scanned values disappear with no warning.	Confirmed	CustomFoodsScreen.kt:447-463
BUG-003	P2	Several logging paths ignore the day on Home, and the search pill always defaults to Breakfast	Food, water or weight meant for a past day lands on today, and food lands under Breakfast at any hour.	Confirmed	NavGraph.kt:183
BUG-004	P2	Plural or extra-word searches return nothing for everyday foods	Typing "bananas", "apples", "onions", "rotis" or "dosas" shows No Foods Found.	Confirmed	LocalFoodDatabaseProvider.kt:357-375
BUG-005	P2	One tap permanently deletes recipes, custom foods, weigh-ins and water logs	A mis-tap beside the Edit icon destroys a recipe or a weigh-in with no check and no undo.	Confirmed	CustomFoodsScreen.kt:329
BUG-006	P2	Water and Weight stay on yesterday after midnight, so a morning log lands on the wrong day	A morning quick-add counts for yesterday, and a morning weigh-in can overwrite yesterday's entry.	Confirmed	WaterScreen.kt:120
BUG-007	P2	Editing current or target weight replaces a strategy the user chose earlier	Saving a new weight can turn Cutting into Maintaining or Bulking, and good days then lose rank points.	Confirmed	PreferencesScreen.kt:198
BUG-008	P2	Changing an old entry's amount recalculates it from the food's current nutrition, not its snapshot	History shifts to the food's new values: setting 1 back to 1 can turn 200 kcal into 250.	Confirmed	FoodDetailScreen.kt:280-282
BUG-009	P2	Some INDB dishes ship on the wrong water basis, so per-100 g values are off by about 2x	Plain dosa logs 381 kcal per 100 g, about double a griddled batter food. Poha and Pesarattu have only the 100 g portion.	Highly Likely	built_in_foods.db:
BUG-010	P2	Leaving the screen during a restore commits the database, skips goals and profile, and shows nothing	The diary changes a few seconds later while goals, strategy and profile stay old, and no message appears.	Confirmed	ImportExportScreen.kt:217-223
BUG-011	P2	Merge import drops the backup's goal history, so imported days leave the rank	After moving to a new phone, rank collapses and past calendar colours are wrong although the food imported fine.	Confirmed	PortabilityRepositoryImpl.kt:385-398
BUG-012	P2	An "Ingredients" or "Allergens" line above the table stops parsing	A clear nutrition table reports Nutrition Info Not Found, or comes back partly filled.	Confirmed	NutritionLabelParser.kt:131-148
BUG-013	P2	Claim lines above the table hijack a nutrient ("20g Protein") or blank it ("100% Whey Protein")	Protein reads 20 instead of 33 with a green tick, or shows Not detected on a label that prints it.	Confirmed	NutritionLabelParser.kt:1119-1134
BUG-014	P2	"Per 100 ml" with a space plus any "%" header loses the per-100 column	Drink labels in the FSSAI layout save the per-serve figure as per 100 ml, about 2x too high.	Confirmed	NutritionLabelParser.kt:931-980
BUG-015	P2	Column anchors come from any word in the image, so columns invert or %RDA is read as per 100 g	Per-100 g values come from the wrong column, for example 26 kcal instead of 520.	Confirmed	SpatialTableDetector.kt:293-324

ID	Priority	Issue	User impact	Confidence	Location
BUG-016	P2	Spatial cells are filled by order, not position, so a neighbouring row or a missed cell shifts values	Sodium can read 1200 mg from the Calcium row, with a green tick.	Highly Likely	SpatialTableDetector.kt:535-543
BUG-017	P2	The unit printed beside a number is ignored: "Sodium 0.4 g" becomes 0.4 mg	Sodium reads 1000 times too low with a green tick, and feeds the rank's food-quality score.	Confirmed	NutritionLabelParser.kt:1099-1105
BUG-018	P3	Recipe servings left blank, 0 or decimal silently become 1	Each logged serving carries the whole pot: 2,000 kcal instead of 800.	Confirmed	RecipesScreen.kt:531
BUG-019	P3	Add Ingredient offers no portion choice, so a typed gram amount multiplies by 100	Typing 250 for 250 g of rice adds 25 kg to the recipe. Catalog cups and spoons cannot be chosen.	Confirmed	RecipesScreen.kt:708
BUG-020	P3	Home's macro rings on a past day use today's macro split	A past day that met its protein goal shows a ring well short of target.	Confirmed	HomeScreen.kt:231-241
BUG-021	P3	On a phone in Light mode, status bar icons are dark on the app's black screen	Clock, battery and notification icons are nearly invisible on every screen.	Highly Likely	MainActivity.kt:48
BUG-022	P3	A failed water, weight, custom food or recipe write crashes the app	On a full disk the app closes with no message, and the entry or form is lost.	Confirmed	WaterScreen.kt:156-174
BUG-023	P3	A double tap in the "+" sheet runs its action twice, doubling a copied meal in the basket	Every copied food appears twice, and logging it doubles the calories if unnoticed.	Highly Likely	AddFoodSheet.kt:63-68
BUG-024	P3	A cancelled search is treated as a failure and flashes Search Error	Normal typing can flash an error card showing a raw coroutine message.	Confirmed	SearchScreen.kt:113-135
BUG-025	P3	After creating a custom food from No Foods Found, the search still says nothing was found	The user may think the save failed, and create a duplicate.	Confirmed	SearchScreen.kt:103-136
BUG-026	P3	Ingredient search can show stale results and can crash the app	The list may not match what was typed, or the app closes and the recipe draft is lost.	Confirmed	RecipesScreen.kt:687-694
BUG-027	P3	Rotating the phone throws the Water and Weight screens back to the route's day	The next entry can go to a different day than the one the user navigated to.	Confirmed	NavGraph.kt:331-335
BUG-028	P3	The weight button says "Edit Today" on every day that has an entry	A user believing they are correcting today overwrites an older weigh-in.	Confirmed	WeightScreen.kt:435-443
BUG-029	P3	The calendar's "today" ring and missed-day count go stale overnight	A wrong today highlight, and a tap on it opens yesterday on Home.	Confirmed	CalendarScreen.kt:135-137
BUG-030	P3	Home's macro ring numbers are cut off at large font sizes	A partial number such as 150/25 for 150 of 250 g on the main dashboard.	Highly Likely	MbMacroRing.kt:67
BUG-031	P3	Per-serving values are saved as per 100 g when "per 100g" appears anywhere on the label	Logging 30 g records about a third of the true value.	Confirmed	NutritionLabelParser.kt:121-129
BUG-032	P3	kJ-only and dual-energy rows ignore columns, so the first column's energy becomes per 100 g	Calories are understated about 2.5 times on Australian and New Zealand labels.	Confirmed	NutritionLabelParser.kt:1060-1097
BUG-033	P3	A decimal comma in serving size turns "12,5 g" into 5 g	Every value is scaled 2.5 times too high on per-serving labels from comma-decimal countries.	Confirmed	NutritionLabelParser.kt:799-802

ID	Priority	Issue	User impact	Confidence	Location
BUG-034	P3	The main dual-column path skips the per-100 g against per-serving cross-check	A value the label itself contradicts gets a green tick.	Confirmed	NutritionLabelParser.kt:178-215
BUG-035	P3	OCR corrections are not applied to basis detection, so a misread "Per 1OOg" inflates values 3.3 times	Values come out too high, under a green-styled "Normalized" note.	Confirmed	NutritionLabelParser.kt:109
BUG-036	P3	Per 1 g drafts can make Save impossible: "4.125" is refused as a thousands separator	A confusing error on a value the user did not type, with Save disabled.	Confirmed	CustomFoodsScreen.kt:430-437
BUG-037	P3	Scanner modals do not block touches or system Back	A stray tap or the Back key discards a reviewed scan.	Highly Likely	NutritionLabelScannerScreen.kt:353-359
BUG-038	P3	Camera bind failures are swallowed, leaving a black preview with no message	A black screen on a device with no usable back camera, and a shutter that does nothing.	Confirmed	NutritionLabelScannerScreen.kt:185-220
BUG-039	P3	A permanently denied camera permission is a dead end, and gallery scanning is blocked too	The Grant button appears to do nothing, and only the manual form is left.	Confirmed	NutritionLabelScannerScreen.kt:118-148
BUG-040	P3	Scanned potassium, calcium and iron are parsed, then dropped with no notice	A new food saves these as "not stated", though the label stated them.	Confirmed	CustomFoodsScreen.kt:472-479
BUG-041	P3	OCR pass agreement and OCR confidence are unused, so the green tick means little	A misread 25 g instead of 2.5 g gets a green tick even if other passes read it correctly.	Confirmed	NutritionValidator.kt:43
BUG-042	P3	Per-serving and blank labels shipped as per-100 g branded values	100 g of these products under-logs by roughly 4x to 20x, and Bingo logs as 0 kcal.	Highly Likely	import_branded_foods.py:67-70
BUG-043	P3	Branded sodium off by large factors passes the 10 g cap	One 250 g Campa drink logs 12,500 mg sodium, and rock salt shows a tenth of what it should.	Highly Likely	import_branded_foods.py:155-156
BUG-044	P3	Open Food Facts zeros for unstated nutrients are stored as real zeros	0.0 g shows instead of "-", and the rank treats these foods as clean.	Highly Likely	import_branded_foods.py:86-88
BUG-045	P3	Main-catalog dish sodium just under the review limit ships unreviewed	One plate of Chicken manchurian logs about 10,740 mg sodium, several times a daily limit.	Highly Likely	built_in_foods.db:
BUG-046	P3	Search tests cover one catalog, but the app merges two	"pistachio" lists a biscuit first, "momo" a branded product, and "dosa" a chutney powder second.	Confirmed	CatalogSearchTests.kt:41
BUG-047	P3	Merge duplicates a water log whose amount was edited after the backup	A day shows 750 ml instead of 500 ml after a Merge.	Confirmed	PortabilityRepositoryImpl.kt:368-379
BUG-048	P3	Leaving during export leaves a 0-byte file named like a valid backup, with no message	The file looks like a backup and fails only when it is needed.	Confirmed	ImportExportScreen.kt:113-150
BUG-049	P3	Export has no out-of-memory handling, so a very large diary crashes the app	Heavy long-term users could never produce a backup, though a backup of that size would restore.	Highly Likely	ImportExportScreen.kt:117-148
BUG-050	P3	Backup file streams are opened on the main thread, which can freeze the UI for cloud files	A frozen screen, possibly an ANR dialog, when restoring from a cloud provider on a slow connection.	Potential	ImportExportScreen.kt:114
BUG-051	P3	A failed figures flow stays dead for the whole process, unlike the rank	Calendar shows an empty grid, or Stats spins forever, with no error until the app restarts.	Potential	FiguresCache.kt:78-92

ID	Priority	Issue	User impact	Confidence	Location
BUG-052	P3	On a fresh install, a second save the same day makes the pending strategy active today	The change applies a day early, and the Pending banner and its cancel link vanish.	Confirmed	GoalsRepositoryImpl.kt:259-281
BUG-053	P3	The catalog's sha256 pins break on a Windows checkout	A fresh clone fails the build with a message saying an unchanged file changed.	Highly Likely	build_catalog.py:389
BUG-054	P4	The Profile "Time zone" setting has no effect, and its UTC default looks real	A traveller or shift worker sets it and nothing changes, with no error for invalid text.	Confirmed	PreferencesScreen.kt:715-722
BUG-055	P4	The calendar treats today differently from the heatmap and the BUG-032 rule	A daily false "Missed" colour, and a green percentage that dips each morning.	Confirmed	CalendarScreen.kt:231-239
BUG-056	P4	A weight logged on a future day vanishes from the list and graph	The entry looks lost, then appears as a real measurement when that date arrives.	Confirmed	WeightScreen.kt:117-124
BUG-057	P4	Any weight edit silently erases that entry's note	A note from an imported backup is lost, invisibly, and drops out of later backups.	Confirmed	WeightScreen.kt:543
BUG-058	P4	The Overwrite warning omits goals and profile, and mentions a "50k food catalog"	Goals change without warning on Overwrite, and the catalog size shown is wrong by a factor of 19.	Confirmed	ImportExportScreen.kt:459-464
BUG-059	P4	The user's own recipes are not searchable, though the README says they are	Typing a saved recipe's name shows No Foods Found and offers to create a custom food.	Confirmed	FoodRepositoryImpl.kt:51-59
BUG-060	P4	Search queries are written to logcat in release builds	What the user searches for appears in logcat and bug reports, against the documented promise.	Confirmed	LocalFoodDatabaseProvider.kt:69
BUG-061	P4	Restore's name fallback can relink an entry to a different food	An old entry can open as a newer food with the same name.	Confirmed	PortabilityRepositoryImpl.kt:556-565
BUG-062	P4	Imported values are not range-checked, and a far-out date makes every later export fail	A hand-edited backup can create an entry no screen shows, which blocks all later exports.	Confirmed	BackupJsonSerializer.kt:142-153
BUG-063	P4	Export is not a consistent snapshot, and stays available during a restore	A valid-looking backup could mix pre-restore diary rows with post-restore foods.	Confirmed	PortabilityRepositoryImpl.kt:48-54
BUG-064	P4	Export does not check import's limits, so one oversized record makes a whole backup unrestorable	"Backup exported successfully!", then a later refusal to restore it.	Confirmed	BackupArchiveManager.kt:51
BUG-065	P4	A deleted catalog version row cannot be re-pinned while the content is unchanged	`--check` says run the build to pin it, the build pins nothing, and the check keeps failing.	Confirmed	build_catalog.py:1294-1318
BUG-066	P4	A variable shadowing bug turns a recipe error into a traceback	A curator gets a TypeError instead of the listed problems. Nothing is written.	Confirmed	build_catalog.py:774
BUG-067	P4	Documentation disagrees with the code in several places	Nothing direct for users, but maintainers and AI agents rely on these documents as contracts.	Confirmed	README.md:51

E. Detailed findings

Within a priority, findings are sorted by users affected, then severity, likelihood, ease of triggering, and how hard the fault is to notice or undo. Evidence blocks are copied from the files at build time, with their real line numbers. Nothing was executed, so "Confirmed" means confirmed by reading and tracing the code (and, for catalog data, by read-only queries).

P0: Critical (0)

No findings at this priority.

P1: High (1)

BUG-001 P1 Label scanner gives every nutrient on a shared line the same number

Confidence Confirmed

Category Bug: wrong data silently accepted

Area Scanner / parser

Location app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1119-1275 (extractNutrient)

What is wrong:

`extractNutrient` uses a keyword only to choose a row. It then collects every number on that row and takes the leftmost one, or the one closest to the column header. Nothing ties a number to the word printed beside it. When a printed line holds two nutrients, the second nutrient receives the first nutrient's value.

How it happens:

An FDA side-by-side panel, common on small packs:

```
Nutrition Facts
Serving size 1 bar (40g)
Amount per serving
Calories 220
Total Fat 12g 15% Total Carbohydrate 15g 5%
Saturated Fat 3g 15% Dietary Fiber 2g 7%
Trans Fat 0g Total Sugars 9g
Cholesterol 0mg 0% Protein 11g
Sodium 120mg 5%
```

- There is no "100g" header, so the spatial path finds no columns and every nutrient uses this function.
- Columns resolve to one value column (per serving) plus %RDA, so the number nearest that column is taken.
- Per serving the result is fat 12 (right), carbs 12 (should be 15), saturated fat 3 (right), fiber 3 (should be 2), sugars 0 (should be 9), protein 0 (should be 11).
- Scaled by 2.5 to per 100 g: 550 kcal, protein 0, carbs 30, fat 30, sugars 0.
- The macro check expects 390 kcal against 550. The difference of 160 is inside the tolerance of 220, so no warning appears and confidence stays HIGH.

Paragraph-style text on small Indian packs fails the same way. `Nutrition per 100g: Energy 450 kcal, Protein 6.0g, Carbohydrate 60g` gives protein 100 and carbs 100, both taken from "100g". The range check only rejects values above 100. `Protein (N x 6.25) 12.0g` gives protein 6.25.

What the user experiences:

The review screen shows Protein 0 g and Sugars 0 g with green ticks next to them. If the user accepts, the saved custom food carries the wrong macros into every diary entry made from it.

Impact:

The whole class of multi-nutrient lines is misread with no warning. Review is always shown, so nothing saves unseen, but the green tick tells the user the value was checked. Wrong values then reach the diary, statistics and rank, and diary snapshots are not corrected when the food is fixed later.

Root cause:

Row-level token collection, with no link between the position of a keyword and the position of a number.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1126-1134

```
1126  for (kw in keywordPatterns) {
1127      val regex = Regex("""\b$kw\b""")
1128      val match = regex.find(textLower)
1129      if (match != null) {
1130          matchedKeyword = kw
1131          break
1132      }
1133  }
1134  if (matchedKeyword.isEmpty()) continue
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1208

```
1208  val valueTokens = rowTokens.filter { !it.isPercent }.sortedBy { it.centerX }
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1245-1254

```
1245  val singleCol = valueCols[0]
1246  // The number under the column's header, not just the leftmost one
1247  val tok = valueTokens.minByOrNull { abs(it.centerX - singleCol.centerX) } ?: valueTokens[0]
1248  if (singleCol.type == ColumnType.PER_100G) {
1249      per100g = tok.value
1250      per100gOp = tok.operator
1251  } else {
1252      perServing = tok.value
1253      perServingOp = tok.operator
1254  }
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1259-1266

```
1259      if (valueTokens.isNotEmpty()) {
1260          perServing = valueTokens[0].value
1261          perServingOp = valueTokens[0].operator
1262      }
1263  } else {
1264      if (valueTokens.isNotEmpty()) {
1265          per100g = valueTokens[0].value
1266          per100gOp = valueTokens[0].operator
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionValidator.kt:143

```
143  val tolerance = max(20.0, calories * 0.40)
```

Assumptions:
OCR merges cells of the same height into one row, which `reconstructSpatialRows` does by vertical overlap.

Recommended fix:
Bind each nutrient to the first non-percent number after its keyword and before the next nutrient keyword. Ignore numbers inside parentheses in the label part. Mark a row needsReview when it holds more than one nutrient keyword.

Regression test:
Parse the FDA sample above and assert carbs 15, sugars 9 and protein 11 per serving. Add a paragraph-style sample and `Protein (N x 6.25) 12.0g`. Build these fixtures with one OCR element per word, as ML Kit reports them.

Related findings:
BUG-013, BUG-016, BUG-034, BUG-041

P2: Medium (16)

BUG-002 P2 A half-filled custom food or recipe is wiped by rotation, a trip to the scanner, or Back

Confidence	Confirmed
Category	Bug: lost input
Area	Custom foods, recipes, scanner hand-off
Location	app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:447-463 (EditCustomFoodScreen) app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:512-517 (EditRecipeScreen) app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:415-421 app/src/main/AndroidManifest.xml:26-31

What is wrong:

Every field of both forms is held in plain `remember`, not `rememberSaveable` or the ViewModel. The activity declares no `configChanges` and no orientation lock, so any configuration change recreates it. Opening the scanner also removes the form from composition, which drops `remember` state. The scanned draft cannot come back either, because it is cleared from the ViewModel as soon as it fills the fields. Neither form has a BackHandler or an unsaved-changes check.

How it happens:

1. Open Create Recipe, add eight ingredients, then rotate the phone, change font size or enter split screen. The form resets to blank, with "4" servings and no ingredients.
2. Open Create Custom Food, type a name and macros, tap "Scan Nutrition Label", then return. Only the scanned numbers appear, and the name is gone, because the parser always sets `foodName = null`.
3. After Apply, rotate while typing the name. Every scanned value and the review banner vanish.
4. Tap Back on either form. The work is discarded with no prompt.

What the user experiences:

The form silently empties. The user has to retype the food, or rebuild the recipe ingredient by ingredient.

Impact:

Affects anyone creating a food or recipe with auto-rotate on, and every user who types a name before scanning. It also makes each crash in BUG-022 and BUG-026 worse, since the draft dies with the process.

Root cause:

Form state is tied to composition. The ViewModel holds only the scanned draft, and only briefly.

Evidence:

`app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:447-453`

```
447  var existingFood by remember { mutableStateOf<CustomFood?>(null) }
448  var name by remember { mutableStateOf("") }
449  var brand by remember { mutableStateOf("") }
450  var servingSize by remember { mutableStateOf("1") }
451  var selectedServingUnit by remember { mutableStateOf(ServingUnit.SERVING) }
452  var customUnitName by remember { mutableStateOf("") }
453  var calories by remember { mutableStateOf("") }
```

`app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:480`

```
480  viewModel.clearScannedDraft()
```

app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:512-517

```
512  var name by remember { mutableStateOf("") }
513  var servingsProduced by remember { mutableStateOf("4") }
514  var ingredients by remember { mutableStateOf<List<RecipeIngredient>>(emptyList()) }
515  var existingRecipe by remember { mutableStateOf<Recipe?>(null) }
516
517  var showAddIngredientDialog by remember { mutableStateOf(false) }
```

app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:415-421

```
415  EditCustomFoodScreen(
416      foodId = foodId,
417      viewModel = viewModel,
418      onScanLabelClick = {
419          navController.navigate(Screen.NutritionLabelScanner.route)
420      },
421      onSaveSuccess = { navController.popIfCurrent(backStackEntry) }
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:594

```
594  foodName = null,
```

Assumptions:

Standard Compose Navigation behaviour: a screen off the top of the stack leaves composition and keeps only saveable state.

Recommended fix:

Hold form state in the per-entry ViewModel, backed by SavedStateHandle, or in `rememberSaveable`, including the ingredient list. Keep the scanned draft until the form is saved or dismissed. Add a discard prompt on Back when the form has unsaved changes.

Regression test:

A StateRestorationTester test: fill the form, recreate, assert fields and banner survive. A navigation test: type a name, push the scanner, pop, assert the name is still there.

Related findings:

BUG-022, BUG-026, BUG-027

BUG-003 P2 Several logging paths ignore the day on Home, and the search pill always defaults to Breakfast

Confidence Confirmed

Category Data integrity: wrong day or meal logged

Area Navigation, Home, Search, Food Detail

Location app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:183 (HomeScreen onOpenSearch)
app/src/main/java/com/macrobase/app/core/navigation/MacroBaseScaffold.kt:190-191 (onLogWater, onLogWeight)
app/src/main/java/com/macrobase/app/domain/model/state/UiStates.kt:43
app/src/main/java/com/macrobase/app/feature/detail/FoodDetailScreen.kt:391-392
app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:350, 376, 393

What is wrong:
Home's meal "+" buttons and the "+" sheet's Search row pass the day on screen, through `dateForNewLog()`. Other paths on the same screen do not:

- The "Search foods to log" pill passes neither date nor meal, so it logs to today under Breakfast.
- The "+" sheet's Log water and Log weight open with no date, so they default to today.
- The Log recipe path always logs to `LocalDate.now()` and starts on Lunch, ignoring the meal chip the sheet just showed.
- A food opened from the Foods tab defaults to Breakfast.

How it happens:
1. View yesterday on Home at 21:00 and tap the search pill.
2. Pick a food and tap "Log 1 food".
3. The food lands on today under Breakfast, and yesterday's page does not change.

With Home on Monday (a past day), tap "+" then "Log weight" and type Monday's weight. It saves to today and, through the unique date index, replaces any entry already there.

What the user experiences:
The food seems to vanish from the day being filled in, and today's totals rise. The only hint is the "When" header further down Food Detail. Users often log the food again, so it ends up on two days.

Impact:
Wrong totals on two days, a wrong streak and rank day, and for weight an overwritten entry.
`BASKET_ARCHITECTURE.md` line 24 says adding from the dashboard uses the selected date.

Root cause:
Each entry point decides its own defaults. There is no shared "logging context" taken from Home.

Evidence:

app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:183

```
183  onOpenSearch = { navController.navigate(Screen.Search.createRoute()) },
```

app/src/main/java/com/macrobase/app/feature/dashboard/HomeScreen.kt:532

```
532  onAddFoodClick = { onNavigateToSearch(meal.type, viewModel.dateForNewLog()?.toEpochDay()) },
```

app/src/main/java/com/macrobase/app/core/navigation/MacroBaseScaffold.kt:178-180

```
178  // On Home the day on screen is used, as Home's own add buttons do; elsewhere today
179  val date = if (onHome) homeViewModel.dateForNewLog() else null
180  navController.navigate(Screen.Search.createRoute(meal.name, date?.toEpochDay()))
```

app/src/main/java/com/macrobase/app/core/navigation/MacroBaseScaffold.kt:190-191

```
190  onLogWater = { navController.navigate(Screen.Water.createRoute()) },
191  onLogWeight = { navController.navigate(Screen.Weight.createRoute(openLog = true)) }
```

app/src/main/java/com/macrobase/app/domain/model/state/UiStates.kt:43

```
43  val targetMealType: MealType = MealType.BREAKFAST,
```

app/src/main/java/com/macrobase/app/feature/detail/FoodDetailScreen.kt:391-392

```
391  val targetDate = basketItem?.date ?: (date ?: LocalDate.now(clock))
392  val targetMealType = basketItem?.mealType ?: (mealType ?: MealType.BREAKFAST)
```

app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:350

```
350  var selectedMeal by remember { mutableStateOf(MealType.LUNCH) }
```

app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:376

```
376  viewModel.addRecipeToBasket(recipe, selectedMeal, LocalDate.now(), qty) {
```

docs/BASKET_ARCHITECTURE.md:24

```
24  - **Adding from Dashboard**: Defaults to the selected dashboard date and meal slot (e.g., if the user taps '+'
on "Lunch", the basket item receives `mealType = LUNCH`).
```

Assumptions:

The search pill counts as adding from the dashboard. `BASKET_ARCHITECTURE.md` line 25 lets generic search default to today, but it suggests Lunch, not Breakfast.

Recommended fix:

Build one helper that returns the date and meal for a new log, and use it in the pill, both sheet actions and the recipe sheet. Use `currentMeal()` as the default for every path that has no meal.

Regression test:

With Home on a past day, assert the pill route and both sheet routes carry that day. With Home on today, assert they carry none. Assert the logged meal follows the clock.

Related findings:

BUG-006, BUG-027, BUG-028

BUG-004 P2 Plural or extra-word searches return nothing for everyday foods

Confidence Confirmed

Category Bug: search returns nothing

Area Search, catalog tooling

Location app/src/main/java/com/macrobase/app/data/provider/LocalFoodDatabaseProvider.kt:357-375, 396-404 (wordSearchSql, wordSearchArgs)
tools/catalog/build_catalog.py:883-935
tools/catalog/synonyms.csv

What is wrong:
Every query word must be the start of a stored word. A query word longer than the stored word ("apples" against " apple ") can never match, and the substring fallback finds no literal "apples" either. All query words are ANDed, so one unmatched word empties the result.

How it happens:
The SQL is `s.terms LIKE '% apples%'`, and the stored terms are " apple seb ". The same happens with "tea with milk", "dal and rice" and "egg fried rice". The auditor's Python port of the search found 35 of 56 sampled plurals return nothing in either catalog. `synonyms.csv` patches a few by hand (eggs, peas, prawns, almonds, grapes).

What the user experiences:
"No Foods Found" for a food the catalog holds, with a button that suggests creating a custom food. "tomatoes" finds only "Stuffed tomatoes", never the plain vegetable.

Impact:
Search is the main route into logging, so this affects most users, and it pushes them into creating duplicate custom foods.

Root cause:
One-directional prefix matching with no plural handling, and strict AND across query words.

Evidence:

app/src/main/java/com/macrobase/app/data/provider/LocalFoodDatabaseProvider.kt:400-403

```
400  val wordStarts = escaped.map { "% $it%" }
401  val wholeWords = escaped.map { "% $it %" }
402  val phrase = escaped.joinToString(" ")
403  return (wordStarts + "%|$phrase|%" + wholeWords + " $phrase%" + wordStarts + limit.toString()).toTypedArray()
```

built_in_foods.db (read-only query): query | food_search matches | substring matches

```
apples | 0 | 0
bananas | 0 | 0
onions | 0 | 0
samosas | 0 | 0
rotis | 0 | 0
dosas | 0 | 0
tomatoes | 1 | 1
apple | 14 | 25
banana | 10 | 10
```

built_in_foods.db (read-only query): stored terms for apple foods

```
 apple jelly seb ki
 apple seb
```

Assumptions:
People type plurals and short phrases. This is common behaviour, not measured here.

Recommended fix:
At build time, add simple plural forms of name words to `terms`. Or at query time, retry a word without its plural ending when the first pass finds nothing. Drop connector words ("and", "with") and relax the AND when a query returns nothing.

Regression test:
Add apples, bananas, onions, samosas, tomatoes and "dal and rice" to `search_cases.csv` as `ok` cases, run through the two-catalog provider.

Related findings:
BUG-046, BUG-025

BUG-005 P2 One tap permanently deletes recipes, custom foods, weigh-ins and water logs

Confidence Confirmed

Category UX: data loss by mis-tap

Area Foods, Recipes, Water, Weight, Basket

Location app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:329
app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:287
app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:466-472
app/src/main/java/com/macrobase/app/feature/water/WaterScreen.kt:330-336
app/src/main/java/com/macrobase/app/feature/basket/BasketScreen.kt:291

What is wrong:
A single tap on the trash icon deletes the row at once. The icon sits directly beside the pencil icon. Whole-meal delete has a confirmation step, so the protection exists, but these paths skip it, and none offers an undo. Basket "Clear" sits directly beside "Log N foods".

How it happens:
In the Foods tab, Recipes view, Weight list or Water list, aim for Edit and hit the trash icon beside it, or brush the icon while scrolling.

What the user experiences:
The row disappears immediately. A recipe's full ingredient list is gone unless a backup exists. A weigh-in also drops out of the trend graph, and the user has to remember the number to re-enter it.

Impact:
Permanent loss of user-entered data from one mis-tap. Diary snapshots are unaffected. Weight and water rows are the record itself, so nothing else can restore them.

Root cause:
Delete callbacks go straight to the repository, and there is no soft delete.

Evidence:

app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:329

```
329  onDelete = { viewModel.deleteCustomFood(item.id) {} }
```

app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:287

```
287  onDeleteClick = { viewModel.deleteRecipe(recipe.id) {} }
```

app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:466-470

```
466  MbIconButton(
467      icon = Ph.Trash.light,
468      contentDescription = "Delete $displayWeight on $displayDate",
469      onClick = { viewModel.deleteWeight(entry.date) },
470      iconSize = 19.dp,
```

app/src/main/java/com/macrobase/app/feature/water/WaterScreen.kt:333

```
333  onClick = { viewModel.deleteWaterEntry(entry.id) },
```

app/src/main/java/com/macrobase/app/feature/basket/BasketScreen.kt:291

```
291  MbButton(text = "Clear", kind = MbButtonKind.Secondary, onClick = { viewModel.clearBasket() })
```

Assumptions:
Treated as a bug, not a deliberate speed choice. Nothing in the docs marks it as intended.

Recommended fix:
Add an undo snackbar that restores the row, or reuse the confirmation step from whole-meal delete.

Regression test:
A ViewModel test that undo restores the row with its id, date, value and ingredients. A UI test that the row stays until the undo window closes.

Related findings:
None

BUG-006 P2 Water and Weight stay on yesterday after midnight, so a morning log lands on the wrong day

Confidence Confirmed

Category Data integrity: wrong day

Area Water, Weight

Location app/src/main/java/com/macrobase/app/feature/water/WaterScreen.kt:120, 156-161 (WaterViewModel)
app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:162, 543 (WeightViewModel)
app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:331-335, 357-361
app/src/main/java/com/macrobase/app/MainActivity.kt:79-83

What is wrong:

Both trackers fix their selected date once. BUG-040 gave Home a `followsToday` flag and `onAppResumed()`, but the trackers got neither. The Weight date range has the same problem, because it is only recomputed when a parameter changes.

How it happens:

1. At 23:00 on Friday, open Home and then the Water or Weight card. The route carries Friday.
2. Leave the app on that screen.
3. On Saturday morning, open it from the launcher or Recents.
4. On Water, tap +250 mL. It logs to Friday.
5. On Weight, the button reads "Edit Today" (BUG-028) and the dialog shows Friday's value. Type 79.6 and save. REPLACE overwrites Friday's entry, and Saturday has none.

If the process was killed overnight, the new ViewModel starts on today, but the route date is applied again and puts it back on Friday.

What the user experiences:

Today's water stays at 0 while yesterday's total grows. Yesterday's real weigh-in is silently replaced. The ribbon shows only the date, never "Today" or "Yesterday", so this is easy to miss.

Impact:

Wrong daily water totals, and lost weight history for anyone who keeps these screens open overnight.

Root cause:

`_selectedDate` is a plain value set at creation, and the rollover logic exists only in `HomeViewModel`.

Evidence:

app/src/main/java/com/macrobase/app/feature/water/WaterScreen.kt:120

```
120  private val _selectedDate = MutableStateFlow(LocalDate.now())
```

app/src/main/java/com/macrobase/app/feature/water/WaterScreen.kt:156-161

```
156  fun addWater(amountMl: Double) {
157      if (amountMl <= 0.0) return
158      viewModelScope.launch {
159          addWaterEntryUseCase(_selectedDate.value, amountMl)
160      }
161  }
```

app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:543

```
543  viewModel.logWeight(uiState.selectedDate, kg)
```

app/src/main/java/com/macrobase/app/MainActivity.kt:79-83

```
79  // A dashboard left on today follows the calendar to the new day (BUG-040)
80  LifecycleResumeEffect(homeViewModel) {
81      homeViewModel.onAppResumed()
82      onPauseOrDispose { }
83  }
```

app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:357-361

```
357  androidx.compose.runtime.LaunchedEffect(dateEpochDayArg) {
358      if (dateEpochDayArg != null) {
359          viewModel.onSelectDate(LocalDate.ofEpochDay(dateEpochDayArg))
360      }
361  }
```

Assumptions:
The app is left on a tracker screen overnight, and the user does not read the ribbon date before logging.

Recommended fix:
Apply the BUG-040 pattern to both tracker ViewModels: a `followsToday` flag, a resume hook, and a recomputed date range. A day the user picked stays picked.

Regression test:
Build each ViewModel with a mutable clock on today, advance a day, call the resume hook, and assert the day moved, writes go to the new day, and `toDate` is the new day. Assert a picked past day stays.

Related findings:
BUG-027, BUG-029, BUG-003, BUG-028

BUG-007 P2 Editing current or target weight replaces a strategy the user chose earlier

Confidence Confirmed

Category Data integrity: silently wrong rank

Area Preferences, goals, rank

Location app/src/main/java/com/macrobase/app/feature/preferences/PreferencesScreen.kt:198, 225-226, 272, 409-415
app/src/main/java/com/macrobase/app/domain/model/GoalStrategyHelper.kt:19-22
app/src/main/java/com/macrobase/app/core/config/RankScoringConfig.kt:111

What is wrong:

`isManuallyOverridden` is true only for a choice made in the current screen session or a change still pending. A strategy chosen earlier and already in force does not count. Any weight edit then re-derives the strategy from the two weights. If target weight is blank, which is allowed, `autoDetectStrategy` returns MAINTAINING, and Save schedules that for tomorrow.

How it happens:

1. The user is on Cutting: maintenance 2,500, daily target 2,000, target weight blank, nothing pending.
2. A later day, they open Preferences and update Current weight only.
3. `onFieldChanged` sets `followsWeights`, so the selected strategy becomes MAINTAINING.
4. Save sees a strategy change and schedules it for tomorrow.

At 2,000 kcal against 2,500 maintenance, the change is 20%. Cutting gives full calorie credit. Maintaining gives zero above 10%, so a good day can cost up to 15 RR and breaks a clean week. A Cutting user with a target weight who edits current weight below target is flipped to Bulking.

What the user experiences:

They recorded a weigh-in and kept eating their cut. From tomorrow, good days show "Calories outside your zone". The strategy chip did move, but nothing says the weight edit caused it, and the Pending line sits far above the Save button.

Impact:

Lost days cannot be repaired by changing the strategy back, because the wrong strategy is written to goal history and the rank replays it.

Root cause:

"Strategy follows the weights" is the default for any weight edit, and a blank target is treated as a request for Maintaining rather than as no signal.

Evidence:

app/src/main/java/com/macrobase/app/feature/preferences/PreferencesScreen.kt:198

```
198  isManuallyOverridden = if (form.isLoaded) form.isManuallyOverridden else goals.scheduledFitnessGoal != null
```

app/src/main/java/com/macrobase/app/feature/preferences/PreferencesScreen.kt:225-226

```
225  private fun autoMatches(form: PreferencesFormState, goals: Goal): Boolean =
226      form.followsWeights && (ProfileField.STRATEGY in form.profileEdited || goals.scheduledFitnessGoal ==
null)
```

app/src/main/java/com/macrobase/app/feature/preferences/PreferencesScreen.kt:272

```
272  followsWeights = form.followsWeights || (weightEdit && !form.isManuallyOverridden)
```

app/src/main/java/com/macrobase/app/feature/preferences/PreferencesScreen.kt:409

```
409  val choseStrategy = edited(ProfileField.STRATEGY) || autoMatches(form, currentGoals)
```

app/src/main/java/com/macrobase/app/feature/preferences/PreferencesScreen.kt:415

```
415  fitnessGoal = if (choseStrategy) form.selectedGoal else currentGoals.scheduledFitnessGoal ?:
currentGoals.fitnessGoal,
```

app/src/main/java/com/macrobase/app/domain/model/GoalStrategyHelper.kt:19-22

```
19  fun autoDetectStrategy(currentWeightKg: Double?, targetWeightKg: Double?): FitnessGoal {
20      if (currentWeightKg == null || targetWeightKg == null) {
21          return FitnessGoal.MAINTAINING
22      }
```

app/src/main/java/com/macrobase/app/core/config/RankScoringConfig.kt:111

```
111  FitnessGoal.MAINTAINING -> ramp(abs(change), 0.05, 0.10, 1.0, 0.0)
```

Assumptions:

Auto-match on a weight edit is meant for users who never chose a strategy. The harm is to those who did.

Recommended fix:

With either weight missing, never change the strategy. Otherwise auto-match only when the user taps "Auto-match to weights" or the stored strategy is still the untouched default, and show a suggestion instead of changing the selection.

Regression test:

Goal(CUTTING, 2500), UserPreferences(currentWeightKg = 80, targetWeightKg = null); edit weight to 79; save. Assert the strategy stays CUTTING, nothing is scheduled, and the weight saved.

Related findings:

BUG-052

BUG-008 P2 Changing an old entry's amount recalculates it from the food's current nutrition, not its snapshot

Confidence	Confirmed
Category	Data integrity: historical snapshot broken
Area	Food Detail, Basket
Location	app/src/main/java/com/macrobase/app/feature/detail/FoodDetailScreen.kt:280-282, 444, 463 (loadFood, onQuantityChange, onServingSelected) app/src/main/java/com/macrobase/app/feature/detail/FoodDetailScreen.kt:423-429 (saveCurrentStateToBasket)

What is wrong:
Edit Mode shows the logged snapshot, but keeps the live food as its calculation base. The first amount change replaces calories, macros and all secondary nutrients with values from the edited custom food or the updated catalog. AGENTS.md section 1.4 requires history to stay immutable, and the code's own comment promises it.

How it happens:

1. "Protein bar" is logged on Monday: quantity 1, 200 kcal.
2. On Wednesday the custom food is edited to 250 kcal. The name and portion stay the same, so `isSameFoodAs` and `canScale` both pass.
3. Open Monday's entry: it shows 200, correct.
4. Tap + to 1.5: it shows 375 (1.5 x 250), not 300.
5. Tap back to 1 and save: the entry becomes 250, though the amount never changed.

The same happens to any entry for a catalog food whose values changed, such as the frying-oil corrections in catalog 2.4.0. In a basket item, a change writes the new nutrition but not the new food, so a later stepper change can make the number fall as the amount rises.

What the user experiences:
History moves to the food's new values with no notice, on an entry the user only meant to adjust.

Impact:
A past day's calories and macros change, and with them that day's calendar colour, statistics and rank replay.

Root cause:
`food` is the looked-up food, and nothing rescales the snapshot in place.

Evidence:

app/src/main/java/com/macrobase/app/feature/detail/FoodDetailScreen.kt:280-282

```
280  val catalogFood = getFoodDetailsUseCase(diaryEntry.food.id)
281      ?.takeIf { it.isSameFoodAs(diaryEntry.food) && it.canScale(it.resolveLoggedServing(diaryEntry.serving)) }
282  val food = catalogFood ?: diaryEntry.food
```

app/src/main/java/com/macrobase/app/feature/detail/FoodDetailScreen.kt:309-310

```
309  // Fill in only the secondary nutrients older versions did not record; calories
310  // and macros stay the logged snapshot even if the food changed since (AGENTS.md 1.4).
```

app/src/main/java/com/macrobase/app/feature/detail/FoodDetailScreen.kt:444

```
444  val calculated = calculateNutritionForServingUseCase(food, serving, qty)
```

Assumptions:
AGENTS.md 1.4 applies to the per-unit basis of an entry the user edits. If recalculating is intended, the code comment and AGENTS.md should say so.

Recommended fix:
In Edit Mode, keep the per-unit snapshot as the base for amount changes, and use live food data only after a deliberate serving change. Keep the item's own nutrition basis in the basket.

Regression test:
Log at 200, change the food to 250, edit quantity to 1.5 and assert 300. Set it back to 1, save, and assert 200.

Related findings:
BUG-061

BUG-009 P2 Some INDB dishes ship on the wrong water basis, so per-100 g values are off by about 2x

Confidence Highly Likely

Category Data correctness

Area Catalog data

Location built_in_foods.db: foods rows INDB:0503, 0117, 0452, 0884, 0931, 0770, 0777, 0776
tools/catalog/corrections.csv

What is wrong:
Griddled and rice dishes carry nearly dry-ingredient values, and boiled-down dishes carry diluted ones. The catalog README records that INDB "loses no water in the pan". Only fried foods receive a water correction, and the build has no energy-density check for cooked dishes. `energy_vs_macros` passes because the macros agree with the energy.

How it happens:
Food Detail opens on the default serving, which is 100 g for every food. Plain dosa therefore logs 381 kcal per 100 g, while Masala dosa is 165 and Semolina dosa 227. Poha and Moong bean dosa have no household portion, so grams are the only route.

What the user experiences:
Where a portion exists, the energy is roughly right (1 dosa = 137 kcal), but its weight is too low. Anyone who weighs their food logs about twice the true energy.

Impact:
Daily totals are wrong for common Indian dishes, for the people most careful about weighing food.

Root cause:
Portion weights were derived from INDB's per-unit energy, which hides the basis error inside the portion. `excess_frying_oil` is the only water-fixing mechanism, and it needs frying oil above 0.

Evidence:

built_in_foods.db (read-only query): name | kcal | protein | carbs | fat (per 100 g) | household portions

```
Tamarind rice (Chintapandu pulihora/Puliyodharai/Puli sadam/Huli anna) | 373.04 | 7.45 | 70.2 | 8.53 | 1 plate=127.8g
Poha | 294.53 | 6.09 | 38.77 | 14.14 | NULL
Plain dosa | 380.91 | 10.34 | 69.31 | 8.43 | 1 dosa=36.1g
Besan bathua chilla/cheela | 373.82 | 11.48 | 48.44 | 18.57 | 1 cheela=30.1g
Namkeen daliya | 295.11 | 8.29 | 57.89 | 7.99 | 1 plate=117.3g
Sprouts upma | 316.25 | 9.5 | 52.12 | 10.44 | 1 plate=95.3g
Moong bean dosa (Pesarattu) | 285.96 | 12.32 | 37.95 | 11.67 | NULL
Maa chaane ki dal | 344.67 | 19.8 | 56.56 | 9.5 | 1 bowl=81.1g
```

built_in_foods.db (read-only query): dosa siblings: name | kcal per 100 g

```
Plain dosa | 380.91
Jowar dosa | 294.35
Moong bean dosa (Pesarattu) | 285.96
Semolina dosa (Suji/Rava dosa) | 227.05
Masala dosa | 164.58
```

Assumptions:
Real griddled batter foods hold far more water than these values imply. The README's own cheela class uses 57 g. The source INDB workbook is not in the repository, so the size of the error is inferred from water by difference and sibling foods.

Recommended fix:
Add a water-only correction kind, or replace these foods through `values_from` or a standard recipe. Add a plausibility check on water by difference for cooked dishes.

Regression test:
A catalog test that Plain dosa, Tamarind rice and Poha fall in a plausible kcal range, and that water by difference for named cooked classes stays above a floor.

Related findings:
BUG-045, BUG-042

BUG-010 P2 Leaving the screen during a restore commits the database, skips goals and profile, and shows nothing

Confidence Confirmed

Category Reliability: partial restore

Area Backup and restore

Location app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:217-223 (executeImport)
app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:208-442 (importUserData)

What is wrong:

The restore runs in `viewModelScope`, and `importUserData` assumes that cancelling it rolls the transaction back. It does not. Inside `withTransaction`, every DAO call runs directly, with no suspension point, so nothing observes the cancellation and the transaction commits. The caller, however, is released at once with CancellationException. The outer `catch (e: Exception)` swallows that, so `restoringSettings` and `onUserDataChanged` never run.

How it happens:

1. Data & backup, pick a file, Overwrite (or Merge on a fresh phone).
2. While "Restoring database transactionally..." shows, press system Back or the header arrow. Nothing blocks this.
3. The navigation entry pops and the ViewModel clears, which cancels the job.
4. The transaction keeps running on Room's executor and commits. Goals, goal history and profile are skipped.

Pressing Back before the transaction thread starts applies nothing. Pressing Back during the database step, which is the long part, gives the broken state.

What the user experiences:

No result dialog and no error. The diary changes on its own while goals, strategy, goal history, name, height and water goal stay as they were. On a fresh phone that means default 2,000 kcal targets. History start becomes today, so every restored day drops out of the rank, and past days are judged against defaults. The widget is not refreshed.

Impact:

A half-applied restore that looks complete. Repeating the restore repairs it, but nothing tells the user to.

Root cause:

A multi-store restore is tied to a screen-scoped coroutine, and it relies on a library cancellation behaviour that does not hold.

Evidence:

app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:217-223

```
217  private fun executeImport(backupData: MacroBaseBackupData, mode: ImportMode) {
218      viewModelScope.launch {
219          _uiState.update { it.copy(isImporting = true, previewData = null) }
220          val result = importUserDataUseCase(backupData, mode)
221          _uiState.update { it.copy(isImporting = false, importResult = result) }
222      }
223  }
```

app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:217

```
217  userDatabase.withTransaction {
```

app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:253

```
253  val settingsNote = restoringSettings {
```

app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:431-437

```
431  } catch (e: Exception) {
432      ImportResult(
433          isSuccess = false,
434          mode = mode,
435          errorMessage = "Import failed transactionally: ${briefErrorText(e, "Database error")}"
436      )
437  }
```

app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:440

```
440  if (result.isSuccess) onUserDataChanged()
```

room-ktx 2.6.1 (library bytecode, read with javap)

```
CoroutinesRoom$Companion.execute: isOpenInternal, inTransaction, then Callable.call directly
withTransaction$transactionBlock$1: beginTransaction, Function1.invoke, setTransactionSuccessful, endTransaction
startTransactionCoroutine: CancellableContinuationImpl, Executor.execute, runBlocking
```

Assumptions:

`koinViewModel()` is scoped to the NavBackStackEntry, which is the default owner. Room is 2.6.1, per `libs.versions.toml`.

Recommended fix:

Run the whole restore (database, settings, `onUserDataChanged`) in an application-lifetime scope or under `withContext(NonCancellable)`, and have the screen observe it. Block Back while restoring, or show the result on return. Rethrow CancellationException in the outer catch.

Regression test:

A Room database with the real transaction executor: hold the transaction with a latch, cancel the ViewModel scope mid-transaction, release, wait. Assert database and goals are both restored or both unchanged. The existing tests inline the executor, so they cannot see this.

Related findings:

BUG-048, BUG-011, BUG-063

BUG-011 P2 Merge import drops the backup's goal history, so imported days leave the rank

Confidence	Confirmed
Category	Data integrity: goal history lost
Area	Backup, goals, rank
Location	app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:385-398 (importUserData, merge branch) app/src/main/java/com/macrobase/app/data/repository/GoalsRepositoryImpl.kt:172-181, 424 app/src/main/java/com/macrobase/app/data/repository/rank/RankRepositoryImpl.kt:320-322

What is wrong:

On a phone where goals were saved, Merge imports the diary rows but ignores `backup.goals`, including `transitions`. The imported days then fall before this phone's first saved day. They are left out of the rank. For Calendar, Consistency and Averages they resolve to this phone's placeholder goal.

How it happens:

- Old phone: Cutting from 2026-03-01 at 1,600 kcal, rating about 1,300.
- New phone: installed 2026-10-20, user saves goals (2,000 kcal, Maintaining).
- 2026-10-25: Merge of the old backup. `historyStart` is 2026-10-20, so every March to October day is dropped from the rank.
- Apr 1 at 1,600 kcal is judged against the 2,000 placeholder: ratio 0.80 gives "Under", where the real result was "On target".

What the user experiences:

Their rank collapses to the bottom tier although the food is all there. Past calendar colours and heatmap percentages are wrong. The import message only says "Your current goals were kept", and the rank page names the day but not why.

Impact:

The only repair is Overwrite, which discards whatever was logged on the new phone.

Root cause:

Merge treats goals as one current setting to keep or replace, while `transitions` is per-day history belonging to the imported days.

Evidence:

app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:385-393

```
385  // Merge keeps this phone's goals and profile. A backup only fills them on a phone
386  // where none were saved yet, such as a fresh install (BUG-016).
387  var keptGoals = false
388  var keptProfile = false
389  val settingsNote = restoringSettings {
390      backupData.goals?.let { g ->
391          if (goalsRepository.hasSavedGoals()) {
392              keptGoals = true
393              skipped++
```

app/src/main/java/com/macrobase/app/data/repository/rank/RankRepositoryImpl.kt:320-322

```
320  val countFrom = historyStart
321  // Every day the rank may count, across seasons; the score card reads these
322  val countedDays = scoreDays(if (countFrom == null) foods else foods.filter { !it.date.isBefore(countFrom) },
today, dayGoals)
```

app/src/main/java/com/macrobase/app/data/repository/GoalsRepositoryImpl.kt:424

```
424  val record = history.lastOrNull { !it.first.isAfter(date) }?.second ?: return@map active
```

app/src/main/java/com/macrobase/app/data/repository/GoalsRepositoryImpl.kt:172-181

```
172  override suspend fun getGoalHistoryStart(): LocalDate? {
173      // Saves today's goal first when no saved day tells from when the goal is known
174      checkAndPromoteScheduledGoal()
175      val today = clock()
176      return parseTransitions(data.first()[Keys.TRANSITIONS_HISTORY])
177          .filter { it.effectiveDate != HISTORY_PLACEHOLDER_DATE }
178          .mapNotNull { runCatching { LocalDate.parse(it.effectiveDate) }.getOrNull() }
179          .filter { !it.isAfter(today) }
180          .minOrNull()
181  }
```

docs/RANKING_SYSTEM.md:123-124

```
123  rank shows the one-time notice anyway. Backups carry the goal history, so a restore keeps the day; a backup
without
124  history (before format 1.2.0, or an empty one) leaves the rank counting from the restore.
```

Assumptions:

The backup has `transitions` (format 1.2.0 or later).

Recommended fix:

In Merge, keep this phone's current goal but merge history records: take backup records dated before this phone's first saved day, replacing the placeholder. Today's and later records stay this phone's.

Regression test:

Phone with history from D; Merge a backup with transitions and diary from D-200. Assert `getGoalHistoryStart()` is D-200, `getGoalForDate(D-100)` is the backup's goal, the current goal is unchanged, and the rank counts the imported days.

Related findings:

BUG-010

BUG-012 P2 An "Ingredients" or "Allergens" line above the table stops parsing

Confidence Confirmed

Category Bug: parse failure

Area Scanner / parser

Location app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:131-148 (parse, row filter)

What is wrong:

"Table found" is set by any row containing nutrition, nutrients, 100g, energy or protein. A later row, or the same row, containing "ingredients" or "allergens" then breaks the loop, and every row below it is dropped.

How it happens:

- "High Protein Muesli" / "Ingredients: Oats, Almonds, Honey" / "Nutrition Information per 100g" / "Energy 420 kcal" / "Protein 20 g" / "Carbohydrate 55 g" / "Fat 12 g". Row 1 sets the flag, row 2 breaks, all nutrients are null, and the result is ParseFailed.
- "INGREDIENTS: Whey Protein Concentrate..." on one row breaks at once.

The spatial path ignores `filteredRows`, so dual-column labels survive. A single-column "per 100 g" table, common on Indian packs, has no such rescue.

What the user experiences:

"Nutrition Info Not Found" on a clear table. Retaking with a tighter crop is the only way out.

Impact:

A loud failure rather than a wrong value, but common for gallery photos, where ingredient text is in frame.

Root cause:

The end-of-table test does not know whether the table has started.

Evidence:

`app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:133-141`

```
133  var foundTableOrNutrient = false
134  for (row in spatialRows) {
135      val lower = row.text.lowercase(Locale.ROOT)
136      if (lower.contains("nutrition") || lower.contains("nutrients") || lower.contains("100g") ||
lower.contains("energy") || lower.contains("protein")) {
137          foundTableOrNutrient = true
138      }
139      if (foundTableOrNutrient && (lower.contains("amino acid profile") || lower.contains("typical amino acid")
|| lower.contains("ingredients") || lower.contains("allergens"))) {
140          break
141      }
```

Assumptions:

The ingredient text is inside the captured area.

Recommended fix:

Stop only after at least one nutrient row with a number has been seen.

Regression test:

The sample above, asserting all four macros are read. `RealLabelImageRegressionTest` has this row but passes only through the spatial path.

Related findings:

BUG-013

BUG-013 P2 Claim lines above the table hijack a nutrient ("20g Protein") or blank it ("100% Whey Protein")

Confidence	Confirmed
Category	Bug: wrong or missing data
Area	Scanner / parser
Location	app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1119-1134, 1194, 1330 (extractNutrient) app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:749

What is wrong:
The non-spatial path scans rows from the top of the image, and the first row containing the keyword wins. Rows are only cut at the end of the table, never before its start. A row whose only number is a percentage returns null straight away, so the real row below is never read.

How it happens:

- Value hijack: "High Protein Bar 20g Protein" / "Nutrition Information Per 100g" / "Energy 420 kcal" / "Protein 33 g" / "Carbohydrate 30 g" / "Fat 15 g" gives protein 20 instead of 33. The macro check stays inside tolerance.
- Blanked value: "100% Whey Protein" / "Nutrition Facts" / "Serving size 1 scoop (30g)" / "Calories 120" / "Protein 24g". Row 1 matches, its only token is a percentage, and line 1330 returns null instead of continuing. Protein is null.
- The same applies to "97% Fat Free" and "0% Added Sugar".

What the user experiences:
A wrong protein value with a green tick, or "Not detected" on a label where protein is plainly printed.

Impact:
A blank macro then saves as 0.0, because the form only requires calories and `Nutrition.proteinGrams` is not nullable.

Root cause:
There is no table-start boundary, and `extractNutrient` returns where it should continue when a row produced no value.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1194
```
1194  if (rowTokens.isEmpty()) continue
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1330
```
1330  return reconcileNutrient(extracted, servingGrams, expectedUnit, basis, row.text)
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1376
```
1376  if (extracted.perServing != null) {
```

app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:749
```
749  protein = proteinInput.value ?: 0.0,
```

Assumptions:
The claim text is inside the captured area. This is most likely with gallery photos and the 5% crop margin.

Recommended fix:
Continue when a row yields no value. Start the nutrient search at the table header or the first anchor row. Tell the user that a blank macro saves as 0.

Regression test:
Both samples, asserting protein 33 and protein 24.

Related findings:
BUG-012, BUG-001

BUG-014 P2 "Per 100 ml" with a space plus any "%" header loses the per-100 column

Confidence Confirmed

Category Bug: wrong column

Area Scanner / parser

Location app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:931-980 (analyzeColumns)
app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:297, 319

What is wrong:
ML Kit returns "100" and "ml" as separate words, so the word-level check for "100 g" can never match. The row-level fallback that would find it runs only when fewer than two column types were found, and "Serve" plus "%RDA" already make two. The per-100 column vanishes, and only the serving column is read.

How it happens:

- Drink: "Per 100 ml Per Serve (200 ml) %RDA" / "Energy 44 88 4%". No gram mass exists and the basis stays PER_100_ML, so 88 kcal is saved for 100 ml, double the true value. Every field shows "Check", but the note says "These values are per 100 ml".
- Solid food: "Nutrients Per 100 g Per Serve (30 g) %RDA" / "Protein 73.7 g 22.1 g 40.9%". Protein is rebuilt as 22.1 x 100/30 = 73.67, which replaces the printed value with a rounded back-calculation.

What the user experiences:
On drink labels the saved value is 2 to 2.5 times too high, under a note that says it is per 100 ml.

Impact:
A very common Indian layout. The error is proportional, so no consistency check can catch it.

Root cause:
Header words are tested one at a time, and the fallback is gated on the type count.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:937-943

```
937  val type = when {
938      text.contains("%") || text.contains("rda") || text.contains("dv") || text.contains("daily value") ->
ColumnType.RDA
939      text.contains("100g") || text.contains("100 g") || text.contains("100gm") || text.contains("100 gm") ||
940              text.contains("100ml") || text.contains("100 ml") -> ColumnType.PER_100G
941      text.contains("serve") || text.contains("serving") || text.contains("portion") -> ColumnType.PER_SERVING
942      else -> null
943  }
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:958-959

```
958  val sBox = row.bounds
959  if (sBox != null && detectedCols.map { it.type }.distinct().size < 2) {
```

app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:319

```
319  } else if (text.contains("100g") || text.contains("100 g") || text.contains("100gm")) {
```

Assumptions:
ML Kit returns "100" and "ml" as separate elements when they are printed with a space. Real ML Kit output was not available here.

Recommended fix:
Join adjacent words ("100" + "g" or "ml") before classifying headers, in both detectors.

Regression test:
A word-level header "Per 100 ml Per Serve (200 ml) %RDA", asserting 44 kcal per 100 ml, plus a solid-food equivalent asserting the printed per-100 value.

Related findings:
BUG-015, BUG-031

BUG-015 P2 Column anchors come from any word in the image, so columns invert or %RDA is read as per 100 g

Confidence Confirmed

Category Bug: wrong column

Area Scanner / spatial table

Location app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:293-324 (detectTableColumns)
app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:582-603 (assignTokensToCells)
app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:680 (reconcileCells)

What is wrong:

BUG-057 fixed only `headerBottomY`. The anchors `elem100` and `elemServ` still take the last matching word anywhere in the OCR. The "size"/"total" exclusion tests single words, but ML Kit elements are single words, so "Serving" in "Serving Size" passes it. The test fixture hides this by using multi-word elements.

How it happens:

- Case A, %RDA read as values: "Serving Size: 30g" / "Nutrient Per 100g %RDA" / "Energy (kcal) 520 26" / "Protein (g) 7.0 14" / "Fat (g) 28 36" / "Carbohydrate (g) 60 20". The serving anchor lands left of the 100 g anchor, so the column order becomes [PER_SERVING, PER_100G]. The bare %RDA numbers are not percent tokens, so 520 goes to serving and 26 to per 100 g. The swap repair needs two proportional rows and only carbs qualifies. The result is 26 kcal, protein 14, fat 36, carbs 20. The macro warning fires, but the values are RDA numbers.
- Case B, silent inversion: "Per 100g Per serving" with rows 400/120 and a left-aligned "Serving suggestion: ..." below the table. `reconcileCells` returns early because grams are null, so calories become 120 per 100 g. Values stay proportional, so no warning fires and confidence is HIGH.

What the user experiences:

Per-100 g values come from the wrong column, in Case B with a green tick.

Impact:

Indian "Per 100g | %RDA" tables with bare percentages are common, so Case A is a realistic path to a badly wrong food.

Root cause:

Anchor selection is not restricted to the header row, and assignment follows anchor order.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:313

```
313  if (text.contains("size") || text.contains("total")) continue
```

app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:319-323

```
319  } else if (text.contains("100g") || text.contains("100 g") || text.contains("100gm")) {
320      elem100 = elem
321  } else if (text.contains("serving") || text.contains("serve") || (servingGrams != null &&
text.contains("${servingGrams.toInt()}g"))) {
322      elemServ = elem
323  }
```

app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:680

```
680  if (servingGrams == null || servingGrams <= 0.0) return adjusted
```

app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:582-584

```
582  if (valueCols.size >= 2 && valueTokens.size >= 2) {
583      val tok0 = valueTokens[0]
584      val tok1 = valueTokens[1]
```

Assumptions:

Case A needs %RDA cells with no "%" sign. Case B needs a stray word left of the 100 g column.

Recommended fix:
Pick anchors from one header row (the topmost row holding a header word), and give bare numbers in the %RDA column to RDA by X position.

Regression test:
Both samples, using word-level elements, asserting per-100 values. Make the BUG-057 test assert columns as well.

Related findings:
BUG-014, BUG-016

BUG-016 P2 Spatial cells are filled by order, not position, so a neighbouring row or a missed cell shifts values

Confidence Highly Likely

Category Bug: wrong column or row

Area Scanner / spatial table

Location app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:535-543, 561-603
app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:68-81 (NUTRIENT_ROW_DEFINITIONS)

What is wrong:

Column `minX` and `maxX` are computed but never used for assignment, although the scanner doc says mapping uses spatial distance. Tokens join the nearest modelled row within 0.95 x 1.3 x line height, about 1.235 line heights. Rows for Calcium, Iron, Potassium, vitamins and MUFA/PUFA have no definition, so on a tight table their numbers join the neighbouring modelled row.

How it happens:

- Row bleed: a Sodium row with "540mg"/"162mg" at top 300, and a Calcium row with "1200mg"/"360mg" at top 336, boxes 30 high. Tolerance is 37 px and pitch 36 px, so the Calcium tokens join Sodium. Sorted by X, "1200mg" can land first: sodium per 100 g = 1200.
- Missed cell: a three-column row where OCR missed the first cell and the %RDA is bare: "Protein [missed] 6.0 12" gives per 100 g = 6.0.

What the user experiences:

A plausible wrong value with a green tick. The same fault can move saturated or trans fat when MUFA/PUFA rows sit beside them.

Impact:

Silent wrong values on dense tables, which is what the spatial path exists to read correctly.

Root cause:

Assignment by token index, a loose vertical tolerance, and no row for unmodelled labels.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:493

```
493  val rowHeight = max(18f, lineBox.height.toFloat() * 1.3f)
```

app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:536-540

```
536      abs(sBox.centerY.toFloat() - row.centerY)
537  }?.takeIf { row ->
538      abs(sBox.centerY.toFloat() - row.centerY) <= (row.height * 0.95f) &&
539      sBox.centerX > (row.labelRight - 15f)
540  } ?: continue
```

app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:582-584

```
582  if (valueCols.size >= 2 && valueTokens.size >= 2) {
583      val tok0 = valueTokens[0]
584      val tok1 = valueTokens[1]
```

docs/NUTRITION_LABEL_SCANNER.md:114

```
114  - Element-to-column mapping assigns values based on horizontal spatial distance rather than raw reading
order.
```

Assumptions:

Line pitch to line height is below 1.235, and label cells are not centre-aligned identically. Real ML Kit geometry was not available here. Test fixtures use a ratio of 1.33, above the threshold.

Recommended fix:
Assign tokens to the column whose X range contains them. Treat any label-like line as a row boundary, even when it is unmodelled, and shrink the tolerance to half the pitch.

Regression test:
The Sodium/Calcium coordinates above, and a missing-cell row with a bare %RDA.

Related findings:
BUG-015, BUG-001, BUG-034

BUG-017 P2 The unit printed beside a number is ignored: "Sodium 0.4 g" becomes 0.4 mg

Confidence Confirmed

Category Bug: wrong unit

Area Scanner / parser

Location app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1099-1105, 1163
app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:170-193
app/src/main/java/com/macrobase/app/feature/scanner/NutritionUnitParser.kt:35

What is wrong:
Each nutrient has a hardcoded expected unit, and the token's own unit is dropped. mg/g/mcg conversion exists only in code that only tests call. The scanner doc promises microgram conversion, but it does not happen.

How it happens:
"per 100g" / "Sodium 0.4g" gives a token of 0.4 with unit g. `RowToken` has no unit field, so the value is stored as 0.4 mg (true value 400 mg). The spatial path does the same. Other cases:

- "Fiber 500 mg" gives 500 g, and fiber has no range check.
- "Salt 450 mg" gives 450 g salt, then 177,165 mg sodium, which the 40,000 mg check does catch.
- "Iron 800 mcg" gives 800 mg.

What the user experiences:
Sodium reads 1000 times too low with a green tick. It is saved, and feeds the rank's food-quality scoring.

Impact:
Silent unit errors of 1000x on labels that print the less common unit.

Root cause:
Unit information is discarded at the token-to-row step.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1099-1105

```
1099  private data class RowToken(
1100      val value: Double,
1101      val operator: ComparisonOperator,
1102      val isPercent: Boolean,
1103      val rawText: String,
1104      val centerX: Float
1105  )
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1163

```
1163  rowTokens.add(RowToken(token.numericValue, op, isPercent, element.text.trim(), cX))
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:183

```
183  sourceUnit = unit,
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionUnitParser.kt:35

```
35  fun convertUnit(value: Double, fromUnit: String, toUnit: String): Double? {
```

app/src/test/java/com/macrobase/app/feature/scanner/NutritionLabelGoldenTests.kt:446

```
446  val convertedMg = NutritionUnitParser.convertUnit(500.0, "mcg", "mg")
```

docs/NUTRITION_LABEL_SCANNER.md:110

```
110  - Microgram units (`mcg`, `µg`, `µg`) are converted to milligrams.
```

Assumptions:
Sodium in grams is a minority format (older UK and EU labels). mg is standard in India and the US.

Recommended fix:
Carry the token's unit, convert to the expected unit, treat "(g)" or "(mg)" in the row label as the unit, and reject tokens with incompatible units.

Regression test:
"Sodium 0.4g" gives 400 mg, "Sodium (g) 0.4" gives 400 mg, "Fiber 500 mg" gives 0.5 g, "Salt 450 mg" gives 0.45 g salt. Run these through `parse()`, not through `convertUnit` directly.

Related findings:
BUG-001

P3: Low (36)

BUG-018 P3 Recipe servings left blank, 0 or decimal silently become 1

Confidence	Confirmed
Category	Bug: wrong nutrition, weak validation
Area	Recipes
Location	app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:531, 543, 565-571 (EditRecipeScreen)

What is wrong:
`servingsProduced` is parsed with a fallback to 1, and no error is shown. The Save button stays enabled.

How it happens:
A dal totals 2,000 kcal for 2.5 servings. Type "2.5", or clear the "4" and leave it blank. The result is 1 serving, so 2,000 kcal per serving instead of 800. "0" also becomes 1.

What the user experiences:
The field shows blank or 2.5 while the recipe saves as 1 serving. The live card does show the larger number, but nothing says why.

Impact:
Every serving logged from that recipe is wrong by the serving count, and diary snapshots keep the wrong value.

Root cause:
`toIntOrNull() ?: 1`, an integer-only model, and no field error.

Evidence:

app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:531
```
531  val servingsCount = (servingsProduced.toIntOrNull() ?: 1).coerceAtLeast(1)
```

app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:543
```
543  val isFormValid = name.isNotBlank() && servingsCount > 0 && ingredients.isNotEmpty()
```

Assumptions:
None for the blank and 0 cases. The number keyboard makes a decimal less likely, not impossible.

Recommended fix:
Show an error and disable Save for blank, 0 or non-integer input, or support fractional servings.

Regression test:
Assert blank, "0" and "2.5" each make the form invalid.

Related findings:
BUG-019

BUG-019 P3 Add Ingredient offers no portion choice, so a typed gram amount multiplies by 100

Confidence Confirmed
Category UX: wrong nutrition
Area Recipes
Location app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:708, 725-734 (add ingredient dialog)

What is wrong:
The dialog takes `food.defaultServing`, which is "100 g" for every catalog food, and offers no picker, although catalog foods have cups, spoons and pieces. The field is labelled only "Quantity" and defaults to "1", with "Serving: 100 g" below it.

How it happens:
Pick Rice and type 250, meaning grams. The ingredient becomes 250 x 100 g = 25 kg.

What the user experiences:
The per-serving total jumps by a very large amount. The user can see it on the live card, but has no clue what went wrong, and has to type 2.5 for 250 g.

Impact:
Recipes are easy to build wrong. When the error is smaller (15 for 15 g of oil becomes 1.5 kg), it is easy to miss.

Root cause:
Food Detail has a serving picker and a total-grams line. The recipe dialog has neither.

Evidence:

app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:708

```
708  selectedServing = food.defaultServing ?: Serving(description = "100 g", gramWeight = 100.0)
```

app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:725-730

```
725  value = qtyInput,
726  onValueChange = { qtyInput = it },
727  label = "Quantity",
728  errorText = if (!ingredientQtyValid) QUANTITY_INPUT_ERROR else null,
729  keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
730  modifier = Modifier.fillMaxWidth()
```

app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:734

```
734  Text(text = "Serving: ${selectedServing?.description ?: "100 g"}", style = AppTypography.Body2)
```

Assumptions:
The user thinks in grams, which is how the "100 g" label reads.

Recommended fix:
Reuse Food Detail's serving picker in this dialog, and show the total grams as the user types.

Regression test:
With a catalog food, assert the dialog offers its servings and that 250 g gives 2.5 x the per-100 g values.

Related findings:
BUG-018, BUG-026

BUG-020 P3 Home's macro rings on a past day use today's macro split

Confidence Confirmed

Category Data integrity: wrong figure (BUG-049 incomplete)

Area Home, Food Detail, goals

Location app/src/main/java/com/macrobase/app/feature/dashboard/HomeScreen.kt:231-241, 741
app/src/main/java/com/macrobase/app/data/repository/DiaryRepositoryImpl.kt:70, 94
app/src/main/java/com/macrobase/app/feature/detail/FoodDetailScreen.kt:180-182, 768

What is wrong:

The summary carries the day's calorie goal, but the percentages come from the live goal, so the ring target mixes two days' goals. `observeGoalsForDates` already returns that day's split, but only `dailyCalorieGoal` is kept. Separately, Food Detail's "% of your daily target" always uses today's goal, even when logging to a past date.

How it happens:

Goal history: 2,000 kcal at 50/25/25 until Oct 15, then 1,600 kcal at 40/35/25. Open Oct 10 on Home. Protein should be 125 g and carbs 250 g. Home shows 2000 x 0.35 / 4 = 175 g and 2000 x 0.40 / 4 = 200 g.

What the user experiences:

Editing the split today changes every past day's rings. A past ring also contradicts the rank row for that day, which scores against 125 g.

Impact:

A wrong picture of past days, with the diary itself unaffected.

Root cause:

`DailyNutritionSummary` has `calorieGoal` but no macro split, so Home fills the gap from the live goal.

Evidence:

app/src/main/java/com/macrobase/app/feature/dashboard/HomeScreen.kt:231-240

```
231  val homeProfile: StateFlow<HomeProfile> = combine(
232      preferencesRepository?.observePreferences() ?: flowOf(UserPreferences()),
233      getGoalsUseCase?.invoke() ?: flowOf(Goal())
234  ) { prefs, goal ->
235      HomeProfile(
236          firstName = prefs.firstName.trim(),
237          unitSystem = prefs.unitSystem,
238          proteinPercentage = goal.proteinPercentage,
239          carbPercentage = goal.carbPercentage,
240          fatPercentage = goal.fatPercentage
```

app/src/main/java/com/macrobase/app/feature/dashboard/HomeScreen.kt:741

```
741  val (proteinTarget, carbTarget, fatTarget) = macroTargetGrams(summary.calorieGoal, profile)
```

app/src/main/java/com/macrobase/app/data/repository/DiaryRepositoryImpl.kt:94

```
94  calorieGoal = goals.dailyCalorieGoal,
```

app/src/main/java/com/macrobase/app/feature/detail/FoodDetailScreen.kt:180-182

```
180  val dailyCalorieGoal: StateFlow<Double> = (
181      getGoalsUseCase?.invoke()?.map { it.dailyCalorieGoal } ?: kotlinx.coroutines.flow.flowOf(0.0)
182  ).stateIn(viewModelScope, kotlinx.coroutines.flow.SharingStarted.WhileSubscribed(5000), 0.0)
```

Assumptions:

The user has changed their split at least once.

Recommended fix:
Add the day's macro percentages or target grams to `DailyNutritionSummary`, and use them for the rings. Give Food Detail the goal for `targetDate`.

Regression test:
Goal history with two splits: `observeDiaryForDate(pastDay)` returns the past split, and ring targets equal that goal's grams.

Related findings:
BUG-011

BUG-021 [P3] On a phone in Light mode, status bar icons are dark on the app's black screen

Confidence Highly Likely

Category UX: system UI

Area App shell

Location app/src/main/java/com/macrobase/app/MainActivity.kt:48
app/src/main/res/values/themes.xml:3-9
app/src/main/java/com/macrobase/app/core/designsystem/Theme.kt:56

What is wrong:
The app is dark only, but `enableEdgeToEdge()` is called with its defaults, which follow the system theme, not the app's. In system Light mode they request dark status icons, and on three-button navigation they also add a light scrim to the navigation bar.

How it happens:
The phone is in Light mode, a common default. Open the app.

What the user experiences:
Status icons are close to invisible on every screen. On three-button phones a pale bar sits under a black app.

Impact:
Every Light-mode user, on every screen. Cosmetic, with no data effect.

Root cause:
Default `SystemBarStyle.auto(...)` in a dark-only app.

Evidence:

app/src/main/java/com/macrobase/app/MainActivity.kt:48

```
48  enableEdgeToEdge()
```

app/src/main/java/com/macrobase/app/core/designsystem/Theme.kt:56

```
56  // MacroBase is dark only by design: one Mentor theme, no light or AMOLED mode
```

app/src/main/res/values/themes.xml:3-6

```
3  <!-- Dark only (the Mentor design). The accent colours the framework date pickers. -->
4  <style name="Theme.MacroBase" parent="android:Theme.Material.NoActionBar">
5      <item name="android:statusBarColor">#000000</item>
6      <item name="android:navigationBarColor">#000000</item>
```

Assumptions:
androidx.activity 1.10.1 default behaviour, known from its public API, not read from source here. `statusBarColor` does not control icon colour.

Recommended fix:
Call `enableEdgeToEdge` with `SystemBarStyle.dark(...)` for both bars.

Regression test:
Robolectric with a light `uiMode`: after `onCreate`, assert `isAppearanceLightStatusBars` and `isAppearanceLightNavigationBars` are false.

Related findings:
None

BUG-022 P3 A failed water, weight, custom food or recipe write crashes the app

Confidence Confirmed

Category Reliability: crash on failed write

Area Water, Weight, Custom foods, Recipes, Home

Location app/src/main/java/com/macrobase/app/feature/water/WaterScreen.kt:156-174
app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:265-281
app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:220-225 (saveCustomFood)
app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:156-165 (saveRecipe)
app/src/main/java/com/macrobase/app/feature/dashboard/HomeScreen.kt:252-260

What is wrong:

These database calls run in `viewModelScope.launch` with no handler. Some have no try/catch at all. The custom food and recipe saves catch the error, reset `saveInFlight` so a retry is possible, then rethrow it, so the process dies and the retry never happens. Food Detail, Basket and Preferences handle the same failure with a visible message.

How it happens:

The phone runs out of space, or SQLite throws an I/O error. The user taps a quick-add or Save. Room throws, the exception reaches the thread's default handler, and the process dies.

What the user experiences:

The app closes with no message. The entry is lost, and the dialog has already closed. It repeats on every retry until space is freed.

Impact:

Rare, but more likely on nearly full budget phones. With BUG-002, it also loses the whole typed form.

Root cause:

Fire-and-forget writes with no error path.

Evidence:

app/src/main/java/com/macrobase/app/feature/water/WaterScreen.kt:156-161

```
156  fun addWater(amountMl: Double) {
157      if (amountMl <= 0.0) return
158      viewModelScope.launch {
159          addWaterEntryUseCase(_selectedDate.value, amountMl)
160      }
161  }
```

app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:265-269

```
265  fun logWeight(date: LocalDate, weightKg: Double, note: String? = null) {
266      viewModelScope.launch {
267          addWeightEntryUseCase(date, weightKg, note)
268      }
269  }
```

app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:220-225

```
220  try {
221      foodRepository.saveCustomFood(customFood)
222  } catch (e: Exception) {
223      saveInFlight = false
224      throw e
225  }
```

app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:162-165

```
162  } catch (e: Exception) {
163      saveInFlight = false
164      throw e
165  }
```

Assumptions:
A real storage failure occurs. The `require()` limits in the use cases are covered by screen validation, so they are not a trigger.

Recommended fix:
Catch failures and rethrow only cancellation. Expose an error in UI state, and keep the dialog or form open until the write succeeds.

Regression test:
A fake repository that throws on write: assert the error is exposed, no exception escapes, and a second Save retries.

Related findings:
BUG-002, BUG-026

BUG-023 [P3] A double tap in the "+" sheet runs its action twice, doubling a copied meal in the basket

Confidence Highly Likely

Category Bug: double submission (BUG-011 sibling)

Area App shell, basket

Location app/src/main/java/com/macrobase/app/core/navigation/AddFoodSheet.kt:63-68 (go)
app/src/main/java/com/macrobase/app/core/navigation/AddFoodSheet.kt:77-95
app/src/main/java/com/macrobase/app/feature/dashboard/HomeScreen.kt:348-356 (copyMealFrom)
app/src/main/java/com/macrobase/app/data/repository/basket/InMemoryBasketRepository.kt:24-28

What is wrong:

`go()` has no "already acting" guard, and rows stay clickable while the sheet hides. Each tap starts a hide job whose completion handler always runs `onDismiss()` and the action, even when that job was cancelled.

How it happens:

1. Open "+" and tap "Copy breakfast from yesterday" twice during the slide-away.
2. The second `hide()` interrupts the first, or the sheet leaves composition. Both completion handlers fire.
3. `copyMealFrom` runs twice, and `addItem` appends with new UUIDs, so the uuid REPLACE guard cannot dedupe.
4. Basket navigation runs twice, so two Basket screens stack. Other rows stack two Search or Water screens.

What the user experiences:

Every food appears twice in the basket, and Back needs extra presses.

Impact:

Doubled calories on the diary day if the user does not notice in the basket.

Root cause:

Action dispatch hangs off `invokeOnCompletion`, with no once-only flag and no check of the cancellation cause.

Evidence:

app/src/main/java/com/macrobase/app/core/navigation/AddFoodSheet.kt:63-68

```
63  fun go(action: () -> Unit) {
64      scope.launch { sheetState.hide() }.invokeOnCompletion {
65          onDismiss()
66          action()
67      }
68  }
```

app/src/main/java/com/macrobase/app/core/navigation/AddFoodSheet.kt:88

```
88  go { homeViewModel.copyMealFrom(yesterday, target, toMeal = target, onDone = {}); onOpenBasket() }
```

app/src/main/java/com/macrobase/app/data/repository/basket/InMemoryBasketRepository.kt:24-28

```
24  override fun addItem(item: BasketItem) {
25      _items.update { current ->
26          current + item
27      }
28  }
```

Assumptions:

The Material 3 sheet stays interactive during its hide animation. Not run on a device.

Recommended fix:
Set a `started` flag in `go` and ignore later calls. Add `launchSingleTop` to these navigations.

Regression test:
A Compose test: two clicks with the clock paused during the hide. Assert `copyMealFrom` ran once and the basket holds one set of foods.

Related findings:
None

BUG-024 P3 A cancelled search is treated as a failure and flashes Search Error

Confidence Confirmed

Category Bug: cancellation treated as failure

Area Search

Location app/src/main/java/com/macrobase/app/feature/search/SearchScreen.kt:113-135 (SearchViewModel.onQueryChange)

What is wrong:

The catch-all `catch (e: Exception)` also catches CancellationException. When a newer keystroke cancels a running query, the old job clears the results, clears the loading flag and sets an error, on top of the newer job's state.

How it happens:

Type a letter, wait out the 250 ms debounce, then type the next letter while that query runs. Typing at about 300 ms per key reaches this window. If the cancelled query finishes after the newer one, for example on the first database open on a slow phone, a valid query can stay on the error card until the next keystroke.

What the user experiences:

A "Search Error" card with text such as "StandaloneCoroutine was cancelled", until the newer query returns.

Impact:

Search looks broken during normal typing.

Root cause:

No CancellationException rethrow, and no check that the job is still the current one.

Evidence:

app/src/main/java/com/macrobase/app/feature/search/SearchScreen.kt:115-118

```
115  delay(DatabaseConfig.SEARCH_DEBOUNCE_MILLIS) // 250ms debounce per UI spec
116
117  try {
118      val results = searchFoodsUseCase(trimmed, limit = 50)
```

app/src/main/java/com/macrobase/app/feature/search/SearchScreen.kt:126-133

```
126  } catch (e: Exception) {
127      _uiState.update {
128          it.copy(
129              searchResults = emptyList(),
130              isLoading = false,
131              errorMessage = e.message ?: "Search failed"
132          )
133      }
```

app/src/main/java/com/macrobase/app/domain/model/state/UiStates.kt:58-59

```
58  isLoading -> SearchContentState.SEARCHING
59  errorMessage != null -> SearchContentState.ERROR
```

Assumptions:

A query takes tens of milliseconds, so the window is reachable. Not measured.

Recommended fix:

Rethrow CancellationException before the generic catch, and show a fixed message rather than `e.message`.

Regression test:

A fake repository suspended on a CompletableDeferred: query "ri", pass the debounce, query "ric", and assert contentState is never ERROR.

Related findings:

BUG-026

BUG-025 P3 After creating a custom food from No Foods Found, the search still says nothing was found

Confidence Confirmed

Category UX: stale state

Area Search

Location app/src/main/java/com/macrobase/app/feature/search/SearchScreen.kt:103-136, 142-156 (SearchViewModel)
app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:240-242

What is wrong:

`searchResults` changes only in `onQueryChange`. The custom food flow updates `customFoods`, which is shown only when the query is empty.

How it happens:

Search "xyz", tap "+ Create Custom Food", and save a food called xyz. The form pops back to Search with the query intact.

What the user experiences:

"No Foods Found" for the food just created.

Impact:

Duplicate custom foods, and a false impression of a failed save.

Root cause:

No re-query when the screen returns or the custom food list changes.

Evidence:

app/src/main/java/com/macrobase/app/feature/search/SearchScreen.kt:145-150

```
145  foodRepository.observeCustomFoods().collect { customList ->
146      _uiState.update {
147          it.copy(
148              customFoods = customList,
149              isInitialLoading = false
150          )
```

app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:240-242

```
240  onCreateCustomFood = {
241      navController.navigate(Screen.EditCustomFood.createRoute())
242  },
```

Assumptions:

None.

Recommended fix:

Re-run the active query when `observeCustomFoods` emits, or open the new food directly after saving.

Regression test:

Query with no hits, emit a matching custom food through the flow, and assert SEARCH_RESULTS.

Related findings:

BUG-004

BUG-026 P3 Ingredient search can show stale results and can crash the app

Confidence Confirmed

Category Bug: race and unhandled exception

Area Recipes

Location app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:687-694 (add ingredient dialog)

What is wrong:

Every keystroke starts an uncancelled, undebounced search on the ViewModel's scope, and the last one to finish wins. Exceptions are not caught, and a blank query leaves the old list on screen.

How it happens:

- Type "ri" then "rice" quickly. If the broader "ri" query finishes last, its results replace those for "rice".
- If the main catalog cannot be opened, `getDatabase()` throws and the app crashes. The Search screen shows an error card for the same failure.

What the user experiences:

The list does not match the typed text, or the app closes and the whole recipe draft is gone.

Impact:

Picking the wrong ingredient from a stale list, or a crash that loses the form.

Root cause:

UI code launches directly on `viewModel.viewModelScope`, with no job tracking.

Evidence:

app/src/main/java/com/macrobase/app/feature/recipes/RecipesScreen.kt:689-693

```
689  if (it.isNotBlank()) {
690      viewModel.viewModelScope.launch {
691          searchResults = viewModel.searchFoodsForIngredient(it)
692      }
693  }
```

app/src/main/java/com/macrobase/app/data/provider/LocalFoodDatabaseProvider.kt:56

```
56  val hits = searchCatalog(databaseManager.getDatabase(), words, trimmed, limit).toMutableList()
```

Assumptions:

The order of finishing depends on the device and the query.

Recommended fix:

Move to SearchViewModel's pattern (one job, a debounce, a cancellation-safe catch), and clear results on a blank query.

Regression test:

A fake repository with controlled completion: finish the older query last and assert the newer results stay. A throwing repository must not crash the test.

Related findings:

BUG-024, BUG-002, BUG-022

BUG-027 P3 Rotating the phone throws the Water and Weight screens back to the route's day

Confidence Confirmed

Category Bug: state reset (BUG-040 sibling)

Area Water, Weight, navigation

Location app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:331-335, 357-361
app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:167-175 (the guarded Home code, for comparison)

What is wrong:
The route date is re-applied each time the destination's composition is rebuilt. Home guards against exactly this with a saved flag. The trackers do not.

How it happens:

1. Home on today, then the Water card.
2. Tap Previous Day twice, to Wednesday.
3. Rotate, or trigger any other configuration change.
4. The ViewModel survives, the `LaunchedEffect` restarts, and the date jumps back to the route's day.
5. A quick-add or Save now hits a different day.

What the user experiences:
The chosen day changes without warning. For weight, the next save can replace that day's entry.

Impact:
Water or weight logged on the wrong day.

Root cause:
`LaunchedEffect(dateEpochDayArg)` has no "applied once" memory.

Evidence:

app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:167-175

```
167  // A day passed in the route is applied once. Coming back to Home (Back, rotation) must
168  // not undo a later day change or the move to a new day after midnight (BUG-040)
169  var appliedDateArg by rememberSaveable { mutableStateOf<Long?>(null) }
170  LaunchedEffect(dateEpochDayArg) {
171      if (dateEpochDayArg != null && dateEpochDayArg != appliedDateArg) {
172          appliedDateArg = dateEpochDayArg
173          homeViewModel.onSelectDate(LocalDate.ofEpochDay(dateEpochDayArg))
174      }
175  }
```

app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:357-361

```
357  androidx.compose.runtime.LaunchedEffect(dateEpochDayArg) {
358      if (dateEpochDayArg != null) {
359          viewModel.onSelectDate(LocalDate.ofEpochDay(dateEpochDayArg))
360      }
361  }
```

Assumptions:
The user changed the day with the arrows after opening the screen.

Recommended fix:
Use the same `rememberSaveable` applied-once guard as Home in both destinations.

Regression test:
Open with a route date, change the day, recreate the activity, and assert the changed day stays.

Related findings:
BUG-006, BUG-002

BUG-028 P3 The weight button says "Edit Today" on every day that has an entry

Confidence Confirmed

Category UX: misleading label

Area Weight

Location app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:435-443 (WeightScreen)

What is wrong:

The label depends only on whether the selected date has an entry, not on whether that date is today.

How it happens:

Open Weight from Home on a past day, or step the ribbon back to any logged day. The button reads "Edit Today". Saving replaces that past day's entry.

What the user experiences:

They believe they are correcting today. The dialog sentence does name the date, but its title is still "Log Body Weight".

Impact:

An overwritten past weigh-in. It also hides BUG-006.

Root cause:

A hard-coded label with no date check.

Evidence:

`app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:435-437`

```
435  val hasSelectedDateWeight = uiState.selectedDateWeight != null
436  MbButton(
437      text = if (hasSelectedDateWeight) "Edit Today" else "Log Weight",
```

Assumptions:

The user trusts the button over the dialog sentence.

Recommended fix:

Build the label from the selected date, and use "Edit Today" only when the date is today.

Regression test:

Extract a label function and assert "Edit Today" appears only when the selected date is today.

Related findings:

BUG-006, BUG-003

BUG-029 P3 The calendar's "today" ring and missed-day count go stale overnight

Confidence	Confirmed
Category	Bug: date rollover (BUG-040 sibling)
Area	Calendar, Statistics
Location	app/src/main/java/com/macrobase/app/feature/calendar/CalendarScreen.kt:135-137, 148, 258 app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:297 app/src/main/java/com/macrobase/app/feature/statistics/StatisticsScreen.kt:338

What is wrong:

`onShown()` runs from `LaunchedEffect`, only when the composable enters composition, not on resume. `today` inside `calculateMonthState` is read only when data emits, and `CalendarScreen` uses plain `collectAsState()`, so it stays subscribed in the background and never restarts.

How it happens:

- Calendar route: leave it open overnight, then return. The ring stays on yesterday, and "Days missed" and "% days of green" are as of yesterday until the diary or a goal changes.
- Stats on the Calendar view: leave it on Oct 31 and return Nov 1. The month stays October, so there is no today cell.

What the user experiences:

A wrong today highlight and a wrong missed count. Tapping the ringed cell opens yesterday on Home, so food can be logged to the wrong date.

Impact:

Misleading figures, and one more route to a mis-dated entry.

Root cause:

Home's BUG-040 fix uses `LifecycleResumeEffect`. The calendar uses a composition-only effect, with no day ticker.

Evidence:

app/src/main/java/com/macrobase/app/feature/calendar/CalendarScreen.kt:135-137

```
135  fun onShown() {
136      if (followsCurrentMonth) _displayedYearMonth.value = YearMonth.now(clock)
137  }
```

app/src/main/java/com/macrobase/app/feature/calendar/CalendarScreen.kt:148

```
148  val today = LocalDate.now(clock)
```

app/src/main/java/com/macrobase/app/feature/calendar/CalendarScreen.kt:258

```
258  val uiState by viewModel.uiState.collectAsState()
```

app/src/main/java/com/macrobase/app/core/navigation/NavGraph.kt:297

```
297  LaunchedEffect(viewModel) { viewModel.onShown() }
```

Assumptions:

The Calendar route stays composed while backgrounded, which is normal.

Recommended fix:

Add a today input to `CalendarViewModel`, refresh it from `LifecycleResumeEffect`, and use `collectAsStateWithLifecycle`.

Regression test:

With a mutable test clock: build state, advance past midnight, call the resume hook, and assert `isToday` and `daysMissed` update, within a month and across a month boundary.

Related findings:

BUG-006, BUG-055

BUG-030 P3 Home's macro ring numbers are cut off at large font sizes

Confidence Highly Likely

Category Accessibility: font scale

Area Home

Location app/src/main/java/com/macrobase/app/core/designsystem/components/MbMacroRing.kt:67, 94-99
app/src/main/java/com/macrobase/app/core/designsystem/Dimensions.kt:58

What is wrong:
"eaten/target" sits in a fixed 78 dp box with `sp` text, `maxLines = 1`, and no ellipsis or auto-size. The default overflow clips.

How it happens:
Assuming an average Jura character width of about 0.55 em, "150/250" needs about 53 dp at 14 sp, 80 dp at font scale 1.5, and 107 dp at 2.0.

What the user experiences:
Part of the number, with no sign that anything is missing.

Impact:
Low-vision users on large fonts see wrong-looking numbers. TalkBack is fine, because the ring has correct screen-reader text.

Root cause:
Scaling text in a fixed-size container, with no shrink or truncation signal.

Evidence:

app/src/main/java/com/macrobase/app/core/designsystem/components/MbMacroRing.kt:94-99

```
94  Text(
95      text = "$eaten/$target",
96      style = AppTypography.Header3.copy(fontSize = 14.sp, letterSpacing = (-0.01).em),
97      color = AppColors.TextPrimary,
98      maxLines = 1
99  )
```

app/src/main/java/com/macrobase/app/core/designsystem/Dimensions.kt:58

```
58  val MacroRing: Dp = 78.dp
```

Assumptions:
The font metric above. The exact threshold may differ.

Recommended fix:
Let the value auto-size within the ring, or scale the ring with font size.

Regression test:
A Compose test at font scales 1.5 and 2.0 with three-digit values, asserting the full text fits inside the ring.

Related findings:
None

BUG-031 P3 Per-serving values are saved as per 100 g when "per 100g" appears anywhere on the label

Confidence Confirmed

Category Bug: wrong basis (BUG-021 sibling)

Area Scanner / parser

Location app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:121-129, 456-467, 759-767
app/src/main/java/com/macrobase/app/domain/model/scanner/ScannerModels.kt:120-122, 191-200

What is wrong:

The final basis comes from text anywhere in the OCR, or from a phantom spatial column, not from what the extracted values are. The BUG-021 guard runs only when `basis == PER_SERVING`.

How it happens:

```
Nutrition Facts
Serving size 1 scoop
Amount per serving
Calories 120
Total Fat 1g 2%
Protein 24g 48%
Typical Amino Acid Profile per 100g of protein
```

The pattern matches the amino line, so the basis is PER_100_G. Columns are [PER_SERVING, RDA], so protein is a per-serving 24 with no grams. The guard at line 456 is skipped, the default output is per 100 g, and Apply saves "100 g = 120 kcal". "Net Wt. 100g" plus any "Serving" word gives the same result through line 122.

What the user experiences:

"Per 100 g" is selected by default. Fields show "Check" with no explanation.

Impact:

This is the outcome `P2PhaseThreeScannerTests.kt` says must never happen. When grams are known, values normalise correctly, so the failure needs a label with no gram weight.

Root cause:

The basis is decided from text before the values are extracted, and it is never checked against them.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:122

```
122  spatialResult.columns.any { it.type == ColumnType.PER_100G } -> NutritionBasis.PER_100_G
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:456

```
456  if (basis == NutritionBasis.PER_SERVING || basis == NutritionBasis.PER_PACKAGE) {
```

app/src/main/java/com/macrobase/app/domain/model/scanner/ScannerModels.kt:120-122

```
120  val canConvertToPer100g: Boolean
121      get() = detectedBasis != NutritionBasis.PER_SERVING && detectedBasis != NutritionBasis.PER_PACKAGE &&
122          detectedBasis != NutritionBasis.PER_100_ML
```

app/src/main/java/com/macrobase/app/domain/model/scanner/ScannerModels.kt:192-195

```
192  val fromMap = per100gValues[key]
193  if (fromMap != null) return fromMap
194  if (primary == null) return null
195  val val100g = primary.normalizedPer100g ?: primary.value
```

Assumptions:

The label gives no gram weight for the serving.

Recommended fix:
After extraction, if primary values are per serving with no gram mass, set the final basis to per serving. Restrict basis text to the table region.

Regression test:
The sample above, asserting detected basis PER_SERVING and output options [PER_SERVING]. Add a "Net Wt. 100g" variant.

Related findings:
BUG-014, BUG-035

BUG-032 P3 kJ-only and dual-energy rows ignore columns, so the first column's energy becomes per 100 g

Confidence Confirmed

Category Bug: wrong column

Area Scanner / parser

Location app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1060-1097 (extractKjEnergy)
app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1014-1048 (extractEnergy)
app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:322-335 (parse)

What is wrong:

These two paths take the first match on the row and label it with the table's basis, with no column logic. Australian and New Zealand labels print per serving first, in kJ.

How it happens:

```
Serving size: 40 g
Quantity per Serving Quantity per 100 g
Energy 680 kJ 1700 kJ
Protein 3.2 g 8.0 g
Fat 4.8 g 12.0 g
Carbohydrate 24.0 g 60.0 g
```

"100 g" with a space turns the spatial path off (BUG-014). `extractKjEnergy` finds "680 kJ" first, with basis PER_100_G, so calories are 162.5 kcal per 100 g. The true value is 406.3. Protein, fat and carbs come out right.

What the user experiences:

Calories are about 2.5 times too low. The macro check fires here, but the wrong number is still pre-filled. With incomplete macros there is no warning.

Impact:

Wrong energy on this format. Those labels are less common for this app's Indian audience.

Root cause:

Energy has its own extraction, outside the column logic.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1075-1077

```
1075  val m = kjPattern.matcher(normalized)
1076  if (m.find()) {
1077      val value = m.group(1)?.let { NutritionDigitCorrector.parseDecimal(it, "kj") }
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:323-328

```
323  val convertedKcal = caloriesKj.value / KJ_TO_KCAL_FACTOR
324  val roundedKcal = (convertedKcal * 10.0).roundToLong() / 10.0
325  calories = ParsedNutrientValue(
326      value = roundedKcal,
327      unit = "kcal",
328      basis = caloriesKj.basis,
```

Assumptions:

The header has a space in "100 g", as the FSANZ template prints it, and ML Kit splits it into two words.

Recommended fix:

Route kJ and dual-energy rows through the same column selection as other nutrients.

Regression test:

The sample above, asserting 406.3 kcal per 100 g and 162.5 per serving.

Related findings:

BUG-014

BUG-033 P3 A decimal comma in serving size turns "12,5 g" into 5 g

Confidence Confirmed

Category Bug: locale parsing

Area Scanner / parser

Location app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:799-802, 867-891 (detectServingInfo)
app/src/main/java/com/macrobase/app/feature/scanner/SpatialTableDetector.kt:222

What is wrong:
All serving regexes accept only "." decimals, and the fallback gram regex matches the digits after the comma.

How it happens:
"Serving size 12,5 g" / "Energy 20 kcal" / "Protein 2 g" / "Carbohydrate 2 g". The first pattern gets 12 with no unit, so there are no grams. The fallback's `\b` before "5" matches "5 g", so grams = 5.0. The result is 400 kcal per 100 g instead of 160, and protein 40 instead of 16.

What the user experiences:
All values wrong, under the note "Normalized from 12.0 (5.0g) serving to per 100 g".

Impact:
Rare for this app. Indian labels use a dot, and a per-100 column stays authoritative when one exists.

Root cause:
Serving parsing does not use `NUMBER_PATTERN` or `parseDecimal`.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:800

```
800  """(?:serving\s*size|per\s*serve|per\s*portion|approx\.?\s*serving)[:\s]*([0-9]+\/[0-9]+|[0-9]+(?:\.[0-9]+)?)?\s*([a-zA-Z]+)?(?:\s*\(([0-9]+(?:\.[0-9]+)?)\s*(g|gm|gms|grams|ml)\))?""",
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:868

```
868  val gramPattern = Pattern.compile("""\b([0-9]+(?:\.[0-9]+)?)\s*(?:g|gm|gms|grams)\b""", Pattern.CASE_INSENSITIVE)
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:876-882

```
876  val gVal = m.group(1)?.toDoubleOrNull()
877  // Ignore 100g if it appears next to "per 100" or is the 100g column header
878  if (gVal == 100.0 && (t.contains("100g") || t.contains("100 g") || t.contains("100gm") || t.contains("per 100"))) {
879      continue
880  }
881  if (gVal != null && gVal > 0.0) {
882      grams = gVal
```

Assumptions:
A comma-decimal label printed per serving only.

Recommended fix:
Use the shared number pattern and parser in all serving regexes.

Regression test:
"Serving size 12,5 g" and "Per portion (12,5 g)" give 12.5 grams.

Related findings:
None

BUG-034 P3 The main dual-column path skips the per-100 g against per-serving cross-check

Confidence Confirmed

Category Validation gap

Area Scanner / parser

Location app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:178-215 (resolveNutrient)
app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1350-1356 (reconcileNutrient)
docs/NUTRITION_LABEL_SCANNER.md:77

What is wrong:
The scanner doc says conflicting columns set needsReview and emit a warning. `crossCheckServingVs100g` has one caller, `reconcileNutrient`, which the spatial path never reaches. On the main route for Indian and EU dual tables, only a times-10 decimal slip can be flagged.

How it happens:
"Serving Size: 30g" / "Nutrients Per 100g Per Serving" / "Total Fat 17.7g 2.3g", where 7.7 was misread as 17.7. Expected serving fat is 5.31 against 2.3, a difference of 3.01 above tolerance. Decimal recovery fails, so the output is fat 17.7 g, HIGH, needsReview false.

What the user experiences:
A green tick on a value the second column disproves.

Impact:
OCR digit misreads pass through on the path meant to catch them.

Root cause:
The cross-check was implemented only in the non-spatial path.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:187

```
187  confidence = ConfidenceLevel.HIGH,
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:1352-1354

```
1352  val matches = NutritionNormalization.crossCheckServingVs100g(effective100g, effectiveServing, servingGrams)
1353  if (!matches) {
1354      needsReview = true
```

docs/NUTRITION_LABEL_SCANNER.md:77

```
77  If values conflict beyond label rounding tolerance, the draft flags `needsReview = true` and emits an
explanatory warning.
```

Assumptions:
Serving grams are known.

Recommended fix:
Run the cross-check on spatial cells as well, set needsReview, and add a warning.

Regression test:
The sample above, asserting `fat.needsReview` is true and a warning is present.

Related findings:
BUG-001, BUG-041

BUG-035 P3 OCR corrections are not applied to basis detection, so a misread "Per 1OOg" inflates values 3.3 times

Confidence Confirmed

Category Bug: wrong basis

Area Scanner / parser

Location app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:109, 112, 155, 287
app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:759-775 (detectBasis)

What is wrong:

`normalizeOcrMisreads` is applied to nutrient rows, but basis and serving detection read the raw text.

How it happens:

"Serving Size: 30g" / "Approx. values Per 1OOg" / "Energy 120 kcal" / "Protein 5 g" / "Carbohydrate 20 g" / "Fat 2 g". "per 1oog" does not match the per-100 pattern, and "serving size" does, so the basis is PER_SERVING. Every value is multiplied by 100/30: 400 kcal instead of 120. The values stay proportional, so no check fires.

What the user experiences:

All values 3.3 times too high.

Impact:

Needs an O/0 misread in the header, which the golden tests show is expected in rows.

Root cause:

The corrected text is not passed to `detectBasis` or the serving parsers.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:109

```
109  val rawBasis = detectBasis(unifiedLines, fullText)
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:155

```
155  val normalizedFullText = normalizeOcrMisreads(fullText)
```

Assumptions:

A single-column table with a serving-size line.

Recommended fix:

Apply the same correction before basis, serving and header detection.

Regression test:

The sample above, asserting 120 kcal per 100 g and no normalisation.

Related findings:

BUG-031

BUG-036 P3 Per 1 g drafts can make Save impossible: "4.125" is refused as a thousands separator

Confidence Confirmed

Category Bug: form validation

Area Scanner hand-off, custom foods

Location app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:430-437 (readFormNumber), 510, 521
app/src/main/java/com/macrobase/app/core/util/DecimalInput.kt:19, 25

What is wrong:

The app itself writes "4.125" into the field, and `readFormNumber` then treats it as "4,125". A one-decimal per-100 value (412.5 kcal, 123.4 mg sodium, any kJ-converted calorie) becomes exactly three decimals per 1 g. A "1/8 cup" serving (0.125) fails the same way.

How it happens:

The label says 412.5 kcal per 100 g. Pick "Per 1 g", then Apply. The form shows 4.125 with "Leave out the thousands separator", and Save is disabled.

What the user experiences:

An error about a separator the user never typed. Changing the value to 4.13 works around it.

Impact:

A blocked save with a misleading message. The same value stored on an existing food also blocks re-saving it.

Root cause:

The check accepts "." groups, and the form cannot tell formatted input from typed input.

Evidence:

app/src/main/java/com/macrobase/app/core/util/DecimalInput.kt:19

```
19  private val THOUSANDS_GROUPED = Regex("""\d{1,3}([.,]\d{3})+""")
```

app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:433

```
433  if (largeAmount && hasThousandsSeparator(trimmed)) return FormNumber(null, "Leave out the thousands separator")
```

app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:510

```
510  val caloriesInput = readFormNumber(calories, largeAmount = true)
```

app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:521

```
521  val sodiumInput = readFormNumber(sodium, largeAmount = true)
```

Assumptions:

None.

Recommended fix:

Refuse only values where a thousands reading is plausible (a non-zero leading group with a comma), or skip the check for values the scanner filled.

Regression test:

`readFormNumber("4.125", largeAmount = true)` returns 4.125, and "2,000" stays refused.

Related findings:

None

BUG-037 P3 Scanner modals do not block touches or system Back

Confidence Highly Likely

Category UX: modal state (BUG-056 sibling)

Area Scanner screen

Location app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelScannerScreen.kt:353-359, 432-440, 475, 886-892

What is wrong:
Only the analysing overlay consumes touches. The review, error, quality, no-text and parse-failed modals are plain Boxes, so a tap on the scrim reaches the shutter, gallery, torch or Back beneath. The shutter stays enabled in these states. There is no BackHandler, so system Back closes the whole scanner rather than the modal.

How it happens:
The review modal is open and the user taps the scrim where the shutter sits. `takePicture` runs, state becomes Capturing, and the draft disappears. Or the user presses Back to close the review and the scan is lost.

What the user experiences:
The reviewed result vanishes.

Impact:
Lost scans, and TalkBack can reach camera controls behind a modal.

Root cause:
Modals are overlays, not dialogs.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelScannerScreen.kt:353

```
353  val isShutterEnabled = !uiState.isAnalyzing && !isCaptureInProgress
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelScannerScreen.kt:435-440

```
435          .pointerInput(Unit) {
436              awaitPointerEventScope {
437                  while (true) awaitPointerEvent().changes.forEach { it.consume() }
438              }
439          }
440  )
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelScannerScreen.kt:886-890

```
886  Box(
887      modifier = Modifier
888          .fillMaxSize()
889          .background(Color(0x99000000))
890          .padding(AppSpacing.md),
```

Assumptions:
The modal is shorter than the screen. Compose hit-testing was reasoned about, not run.

Recommended fix:
Disable the underlying controls while a modal is open, add a scrim touch blocker, and add a BackHandler mapped to Retake.

Regression test:
A Compose test tapping the scrim position in review state, asserting no capture and that the draft is kept.

Related findings:
None

BUG-038 P3 Camera bind failures are swallowed, leaving a black preview with no message

Confidence Confirmed

Category Reliability: swallowed failure

Area Scanner screen

Location app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelScannerScreen.kt:185-220

What is wrong:

Any bind failure only reaches `printStackTrace`. The manifest marks the camera as not required, so a device with no back camera can install the app. If the provider itself fails, `imageCaptureUseCase` stays null and the shutter silently does nothing.

How it happens:

Open the scanner on a device with no back camera, or where the provider fails to initialise.

What the user experiences:

A black screen and either no response or a late capture error. The gallery button still works.

Impact:

A dead end with no explanation on a small set of devices.

Root cause:

The ViewModel's CameraError state is never fed from the bind step.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelScannerScreen.kt:206

```
206  imageCaptureUseCase = imageCapture
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelScannerScreen.kt:217-218

```
217  } catch (e: Throwable) {
218      e.printStackTrace()
```

Assumptions:

None.

Recommended fix:

Report bind failures to the ViewModel, show an error that offers the gallery, and disable the shutter until bound.

Regression test:

Extract the bind error into a callback, and test that it sets CameraError.

Related findings:

BUG-039

BUG-039 P3 A permanently denied camera permission is a dead end, and gallery scanning is blocked too

Confidence Confirmed

Category UX: permissions dead end

Area Scanner screen

Location app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelScannerScreen.kt:118-148, 819-830

What is wrong:

After "Don't ask again", "Grant Camera Permission" returns false at once. There is no link to app settings. The whole screen returns early, so the gallery picker, which needs no camera permission, cannot be reached. The permission is not rechecked on resume either.

How it happens:

Deny twice, then open the scanner.

What the user experiences:

The button appears to do nothing.

Impact:

The scanner is unusable for these users, even for photos they already have.

Root cause:

The denied branch is a full-screen return, with no rationale or settings path.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelScannerScreen.kt:141-147

```
141  if (!hasCameraPermission) {
142      CameraPermissionDeniedContent(
143          onRequestPermission = { permissionLauncher.launch(Manifest.permission.CAMERA) },
144          onNavigateBack = onNavigateBack,
145          modifier = modifier
146      )
147      return
```

Assumptions:

None.

Recommended fix:

Add an "Open settings" action when no rationale is available, add "Choose a photo" to this screen, and recheck on resume.

Regression test:

A Compose test of the denied state, asserting the gallery action exists.

Related findings:

BUG-038

BUG-040 P3 Scanned potassium, calcium and iron are parsed, then dropped with no notice

Confidence Confirmed

Category Data loss: scanned values dropped

Area Scanner hand-off, custom foods

Location app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:472-479, 756-759

What is wrong:

The custom food stores these fields, but the form ignores them from the draft. Trans fat and cholesterol are parsed as well, but `CustomFood` has no column for them.

How it happens:

Scan a label with "Calcium 120mg", Apply, Save. `calciumMg` is null.

What the user experiences:

Data is silently missing. It is not wrong.

Impact:

Minor: these minerals are not shown in the review modal, so the user is never told.

Root cause:

The form fills 8 nutrients, and carries the other 3 only from an existing food.

Evidence:

app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:479

```
479  draft.sodium?.let { sodium = formatQuantityForInput(it.value) }
```

app/src/main/java/com/macrobase/app/feature/customfoods/CustomFoodsScreen.kt:756-759

```
756  // Not editable in this form: carry the stored values through
757  potassium = existingFood?.nutritionPerServing?.potassiumMg,
758  calcium = existingFood?.nutritionPerServing?.calciumMg,
759  iron = existingFood?.nutritionPerServing?.ironMg,
```

Assumptions:

None.

Recommended fix:

Carry these three from the draft through to `saveCustomFood`.

Regression test:

Apply a draft with calcium, and assert the saved food has it.

Related findings:

None

BUG-041 [P3] OCR pass agreement and OCR confidence are unused, so the green tick means little

Confidence Confirmed

Category Validation gap

Area Scanner / OCR ensemble

Location app/src/main/java/com/macrobase/app/feature/scanner/NutritionValidator.kt:43 (resolveCandidates)
app/src/main/java/com/macrobase/app/feature/scanner/OcrEnsemble.kt:143
app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelScannerScreen.kt:1062-1068

What is wrong:

`resolveCandidates`, which compares values across passes, has no callers in main or test. Only the best pass is parsed, ML Kit confidence is never read by the parser, field confidence is hardcoded HIGH, and the green tick shows whenever needsReview is false.

How it happens:

Single column, "Protein 2.5g" read as "25g" in the winning pass while other passes read 2.5. Nothing compares them. If macros stay within 40%, the review shows 25 g with a green tick.

What the user experiences:

The tick reads as verified.

Impact:

Review is always shown, so this does not skip review, but it steers attention away from wrong values.

Root cause:

The ensemble stops at picking a pass.

Evidence:

app/src/main/java/com/macrobase/app/feature/scanner/OcrEnsemble.kt:143

```
143  val bestPass = scoredPasses.maxByOrNull { it.overallQualityScore } ?: scoredPasses.first()
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionValidator.kt:43

```
43  fun resolveCandidates(
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelParser.kt:187

```
187  confidence = ConfidenceLevel.HIGH,
```

app/src/main/java/com/macrobase/app/feature/scanner/NutritionLabelScannerScreen.kt:1062-1068

```
1062  } else {
1063      Icon(
1064          imageVector = Icons.Default.CheckCircle,
1065          contentDescription = null,
1066          tint = AppColors.Success,
1067          modifier = Modifier.size(16.dp)
1068      )
```

Assumptions:

How often passes disagree was not measured.

Recommended fix:

Parse each pass, or at least compare numbers per row, and mark disagreements for review. Drop the green tick, or tie it to real agreement.

Regression test:

An orchestrator test with a fake engine returning 25g and 2.5g across passes, asserting needsReview.

Related findings:

BUG-001, BUG-034

BUG-042 P3 Per-serving and blank labels shipped as per-100 g branded values

Confidence Highly Likely

Category Data correctness

Area Catalog data, importer

Location tools/catalog/import_branded_foods.py:67-70, 76-83, 164-169
branded_foods.db

What is wrong:

Some products hold per-serving numbers, and one snack holds all zeros. Per-serving macros still agree with their own energy, so the Atwater check passes. The kcal minimums are keyword lists that omit chocolate, dates, masala and candy. Category ranges apply only when one exact tag is present, and nothing rejects an all-zero food unless the name hits `HAS_PROTEIN`.

How it happens:

Search "Milkybar" and pick "Milkybar Milk Chocolate", or any product below.

What the user experiences:

63 kcal per 100 g for a chocolate that the same catalog lists at 497.

Impact:

Under-logged calories on the affected products. Scan counts are low, so reach is small.

Root cause:

There is no energy-density check except where a name keyword or tag happens to apply.

Evidence:

branded_foods.db (read-only query): name | kcal | protein | carbs | fat (per 100 g)

```
Bingo Hot & Spicy Korean Style | 0.0 | 0.0 | 0.0 | 0.0
Chupa Chups Sour Belt | 36.5 | 0.25 | 8.13 | 2.3
Everest Pani Puri Masala | 17.25 | 0.44 | 3.02 | 0.36
Good Life Arabian Dates | 312.0 | 2.4 | 72.7 | 0.3
Lion Layina Dates | 83.3 | 0.4 | 20.3 | 0.06
Milkybar Milk Chocolate | 63.0 | 1.3 | 6.0 | 3.6
Nestle Milkybar | 497.0 | 11.1 | 51.6 | 27.3
Nutraj Premium Arabian Dates | 83.5 | 0.5 | 19.6 | 0.4
Sobisco Desire Strawberry Cream | 83.93 | 0.85 | 9.4 | 4.77
```

tools/catalog/import_branded_foods.py:76-83

```
76  MINIMUM_KCAL = [
77      (re.compile(r"\b(ghee|oil|vanaspati)\b"), 650),
78
(re.compile(r"\b(chips|wafers|crisps|namkeen|bhujia|mixture|sev|chivda|chakli|kurkure|puffs|nachos|khakhra)\b"),
350),
79      (re.compile(r"\b(biscuits?|cookies?|crackers?|rusks?)\b"), 350),
80      (re.compile(r"\b(muesli|granola|oats|cornflakes|flakes)\b"), 300),
81      (re.compile(r"\b(bars?|whey|protein powder|makhana|nuts?|almonds?|cashews?|peanuts?|seeds?|dry ?
fruits?)\b"), 300),
82      (re.compile(r"\b(noodles?|pasta|vermicelli|macaroni|flour|atta|maida|besan|suji|rava|jaggery|milk
powder|dairy whitener)\b"), 300),
83  ]
```

Assumptions:

None beyond typical energy for these food types. Milkybar is confirmed by the catalog's own second entry.

Recommended fix:
Extend `MINIMUM_KCAL`, reject all-zero products unless they are water, salt or diet drinks, and list known bad rows in `branded/exclusions.csv`.

Regression test:
Importer tests for "Milk Chocolate" at 63 kcal with no chocolate tag, and for an all-zero snack.

Related findings:
BUG-043, BUG-044

BUG-043 P3 Branded sodium off by large factors passes the 10 g cap

Confidence	Highly Likely
Category	Data correctness: units
Area	Catalog data, importer
Location	tools/catalog/import_branded_foods.py:155-156 tools/catalog/build_branded.py:35-36

What is wrong:
The only sodium check is `> 10_000` mg, and `sodium_level` is deliberately not required for branded foods. Unit-entry errors survive.

How it happens:
Log Campa Lemon Flavoured or Ensure Vanilla Flavour.

What the user experiences:
Sodium up to several hundred times what the product holds, and the rank's sodium input is distorted.

Impact:
Distorted sodium totals and food-quality scores on the affected products.

Root cause:
A single cap sized for pickles is applied to every kind of food.

Evidence:

branded_foods.db (read-only query): name | kcal | sodium mg per 100 g

```
Abbott Ensure Diabetes Care Vanilla Delight Flavour | 435.0 | 324.0
Campa Lemon Flavoured | 46.0 | 5000.0
Coca Cola Sprite | 40.0 | 14.0
Ensure Vanilla Flavour | 410.0 | 6000.0
Natural Tattva Himalayan Rock Salt | 0.0 | 3965.1
```

tools/catalog/import_branded_foods.py:155-156

```
155  if (values["sodiumMgPer100g"] or 0) > 10_000:
156      return None, "more than 10 g sodium per 100 g"
```

Assumptions:
Real labels are near sibling products. Pure salt is about 39,300 mg sodium per 100 g.

Recommended fix:
Add category or name sodium ceilings (drinks, flours, sweets), and exclude the rows above.

Regression test:
An importer test that a drink at 5000 mg is dropped while a pickle at 5000 mg stays.

Related findings:
BUG-042

BUG-044 P3 Open Food Facts zeros for unstated nutrients are stored as real zeros

Confidence Highly Likely

Category Nullability (AGENTS.md 2.4)

Area Catalog data, importer

Location tools/catalog/import_branded_foods.py:86-88, 170-171
app/src/main/java/com/macrobase/app/domain/model/rank/RankModels.kt:201-202

What is wrong:

The catalog README notes Open Food Facts often stores unfilled values as 0, but only the `SALTY` name list acts on it, and that list is matched against sodium alone.

How it happens:

Open a peanut butter or a salty biscuit from the branded catalog.

What the user experiences:

"0.0 g" or "0.0 mg" appears instead of "-".

Impact:

The rank's `KnownAmount.of` counts 0 as known, so a food with unfilled values scores as clean. Some zeros are real, so the counts below are upper bounds.

Root cause:

Zero-versus-missing logic exists for one nutrient and a fixed word list.

Evidence:

branded_foods.db (read-only query): products with sat fat 0 and fat >= 5 | sugar 0 and carbs >= 30 | fibre 0 and carbs >= 40 | sodium 0

```
13 | 55 | 84 | 123
```

tools/catalog/import_branded_foods.py:170-171

```
170  if values["sodiumMgPer100g"] == 0 and SALTY.search(named):
171      values["sodiumMgPer100g"] = None  # not filled in: a salted snack has some
```

Assumptions:

Implausible zeros are unfilled fields. The Open Food Facts snapshot is not in the repository, so this could not be confirmed row by row.

Recommended fix:

Treat a 0 as missing when a related macro makes it implausible, and use category tags as well as names.

Regression test:

An importer test that saturated fat 0 with 30 g fat becomes blank, while a cooking oil's sodium of 0 stays 0.

Related findings:

BUG-042

BUG-045 P3 Main-catalog dish sodium just under the review limit ships unreviewed

Confidence Highly Likely

Category Data correctness

Area Catalog data

Location built_in_foods.db: foods INDB:0848, INDB:0856, INDB:0056
tools/catalog/build_catalog.py:92

What is wrong:

Chicken manchurian has 2,414.77 mg sodium per 100 g, 85 mg below the threshold that would have forced a review. `SODIUM_LIMIT_MG = 2500` does not distinguish a dish from a pickle.

How it happens:

Log 1 plate (444.7 g) of Chicken manchurian.

What the user experiences:

A sodium total many times a daily limit from a single plate.

Impact:

Sodium totals are distorted. Veg manchurian is 144 mg, and `exclusions.csv` already removes sweet-and-sour dishes for this same fault at 3,100 to 4,600 mg.

Root cause:

The same INDB soy-sauce or salted-water fault as dishes already excluded, but this one falls below the threshold.

Evidence:

built_in_foods.db (read-only query): name | kcal | sodium mg per 100 g | portions

```
Poached egg | 123.87 | 1163.07 | 1 egg=56.0g
Chicken manchurian | 142.34 | 2414.77 | 1 plate=444.7g
Cajun chicken | 184.94 | 1740.13 | 1 chicken breast=167.5g
```

tools/catalog/build_catalog.py:92

```
92  SODIUM_LIMIT_MG = 2500.0
```

Assumptions:

Salted cooking water or soy sauce is over-counted in these INDB recipes.

Recommended fix:

Override or exclude these foods, and set a lower sodium limit for dishes than for pickles and spice mixes.

Regression test:

A Python check that no non-pickle, non-spice food exceeds a dish sodium limit without an exception row.

Related findings:

BUG-009

BUG-046 P3 Search tests cover one catalog, but the app merges two

Confidence Confirmed

Category Test gap and ranking

Area Search, tests

Location app/src/test/java/com/macrobase/app/CatalogSearchTests.kt:41
app/src/main/java/com/macrobase/app/core/di/DiModules.kt:124
app/src/main/java/com/macrobase/app/data/provider/LocalFoodDatabaseProvider.kt:68, 382-386
tools/catalog/build_catalog.py:433-438, 938-944

What is wrong:
All `ok` cases pass on the main catalog alone, but 2 of 242 cases fail once branded results merge in as production does. Merging by whole-word tier lets a branded product beat a generic food stored in the plural. Slash parts of a branded name also become exact names.

How it happens:
A singular query matches a plural main name only as a prefix (tier 2), while a branded name holding the word whole is tier 1. `exact_names` treats each slash part of a branded name as a full name.

What the user experiences:
Generic foods fall below branded products for common words.

Impact:
Worse ranking on everyday queries, which the test suite cannot see.

Root cause:
Tier scores are compared across catalogs without the priority signal, and the test harness omits the branded manager.

Evidence:

app/src/test/java/com/macrobase/app/CatalogSearchTests.kt:41

```
41  provider = LocalFoodDatabaseProvider(BuiltInDatabaseManager(context))
```

app/src/main/java/com/macrobase/app/core/di/DiModules.kt:124

```
124  single<FoodDataProvider> { LocalFoodDatabaseProvider(get(), get(named(BRANDED_CATALOG))) }
```

app/src/main/java/com/macrobase/app/data/provider/LocalFoodDatabaseProvider.kt:68

```
68  val foods = hits.sortedBy { it.tier }.take(limit).map { it.food }
```

Assumptions:
None. The merged results come from a Python port of the search SQL run against the shipped files, not from the app.

Recommended fix:
Run `search_cases.csv` through the two-catalog provider. Break ties by main-catalog priority or plural-aware whole words, and skip slash splitting for branded foods.

Regression test:
Build the `CatalogSearchTests` provider with both managers.

Related findings:
BUG-004

BUG-047 P3 Merge duplicates a water log whose amount was edited after the backup

Confidence Confirmed

Category Data integrity: merge semantics

Area Backup and restore

Location app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:368-379
app/src/main/java/com/macrobase/app/feature/water/WaterScreen.kt:450

What is wrong:
The identity key includes `amountMl`, but editing a log keeps its timestamp and changes only the amount. An edited log therefore does not match its own backup copy.

How it happens:
1. Back up with a log of 250 ml.
2. Edit that log to 500 ml.
3. Merge the backup: a second row of 250 ml with the same timestamp is inserted.

What the user experiences:
That day shows 750 ml instead of 500 ml. It reports as "imported", with no conflict, and each edited log adds one extra row.

Impact:
Inflated water totals.

Root cause:
Water rows have no stable id, and the key uses a mutable field.

Evidence:

app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:369-373

```
369      .map { Triple(it.dateEpochDay, it.timestamp, it.amountMl) }.toSet()
370  val toInsertWater = mutableListOf<WaterLogEntity>()
371  for (dto in backupData.waterEntries) {
372      val key = Triple(dto.dateEpochDay, dto.timestamp, dto.amountMl)
373      if (!existingWater.contains(key)) {
```

app/src/main/java/com/macrobase/app/feature/water/WaterScreen.kt:450

```
450  viewModel.updateWaterEntry(entry.copy(amountMl = ml))
```

docs/BACKUP_FORMAT.md:215

```
215  - **Water Logs**: Matched by `(dateEpochDay, timestamp, amountMl)`. Duplicate timestamps skipped; new logs
inserted.
```

Assumptions:
None.

Recommended fix:
Match on `(dateEpochDay, timestamp)` and keep the phone's row on conflict. Reconsider this if water rows gain a uuid.

Regression test:
Merge a backup holding (day, ts, 250) into a phone holding (day, ts, 500), and assert one row and a 500 total.

Related findings:
None

BUG-048 P3 Leaving during export leaves a 0-byte file named like a valid backup, with no message

Confidence Confirmed

Category Reliability: silent failure

Area Backup and restore

Location app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:113-150 (exportBackup)
app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:250-264

What is wrong:
ACTION_CREATE_DOCUMENT has already created the file when the picker returns, and the archive is then built in `viewModelScope`. Cancelling during the build means `outputStreamSupplier()` is never called, so the file stays empty. A read failure leaves the same empty file, though that case does show an error.

How it happens:
Export, choose a location, then press Back before the banner appears. The window grows with diary size (database reads, serialising, SHA-256, zip).

What the user experiences:
`MacroBase_Backup_YYYY-MM-DD.zip` exists at 0 bytes and looks like a backup. Restoring it later gives "Selected backup archive is empty or invalid."

Impact:
A user believes they hold a backup and finds out they do not at the moment they need it. The code comment treats leaving as harmless because restore rejects the file.

Root cause:
Export is tied to the screen's lifetime.

Evidence:

app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:118-119

```
118  val archive = ByteArrayOutputStream().also { exportUserDataUseCase.writeBackupArchive(it) }.toByteArray()
119  val stream = outputStreamSupplier()
```

app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:136-138

```
136  } catch (e: CancellationException) {
137      // Leaving the screen is not a failure; an unfinished file fails validation on restore
138      throw e
```

Assumptions:
The provider creates the document at pick time, which is standard SAF behaviour.

Recommended fix:
Run export in an app scope so it finishes after the user leaves. Failing that, on cancel or pre-write failure, delete the document only if its size is 0, since an empty document cannot be an older backup.

Regression test:
Hold the fake `writeBackupArchive`, clear the ViewModel, and assert the export still completes, or that a 0-size destination is discarded while a non-empty one is kept.

Related findings:
BUG-010, BUG-049

BUG-049 P3 Export has no out-of-memory handling, so a very large diary crashes the app

Confidence Highly Likely

Category Reliability: crash on large data

Area Backup and restore

Location app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:117-148
app/src/main/java/com/macrobase/app/data/portability/BackupArchiveManager.kt:71-124

What is wrong:
BUG-047 added OOM handling to validation and restore, but export catches only `Exception`. It also has the highest peak memory: all entities and DTOs at once, one String per file, each file encoded twice, and the whole archive in a ByteArrayOutputStream that is then copied.

How it happens:
Export on a phone with an ordinary heap limit and a diary of the size the code plans for: five years at 30 entries a day is about 42 MB of JSON and 55,000 records.

What the user experiences:
The app crashes during "Creating backup archive...".

Impact:
No backup is possible for exactly the users with the most to lose. New installs are far from this size.

Root cause:
The memory guard was applied to one direction only, and export buffers everything.

Evidence:

app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:139

```
139  } catch (e: Exception) {
```

app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:189

```
189  } catch (e: OutOfMemoryError) {
```

app/src/main/java/com/macrobase/app/data/portability/BackupArchiveManager.kt:72

```
72  val diaryJson = BackupJsonSerializer.serializeDiaryEntries(data.diaryEntries)
```

app/src/main/java/com/macrobase/app/data/portability/BackupArchiveManager.kt:81

```
81  PortabilityConfig.FILE_DIARY to calculateSha256(diaryJson),
```

Assumptions:
The threshold depends on device heap and was not measured. The missing catch is confirmed.

Recommended fix:
Catch OutOfMemoryError with a message, and stream records through the ZIP and digest instead of building Strings.

Regression test:
A fake `writeBackupArchive` throwing OutOfMemoryError: assert an error message, `isExporting` false, and no crash.

Related findings:
BUG-048, BUG-064

BUG-050 P3 Backup file streams are opened on the main thread, which can freeze the UI for cloud files

Confidence	Potential
Category	Reliability: ANR
Area	Backup and restore
Location	app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:114, 119, 153, 156, 257-259, 270-272

What is wrong:

`openInputStream` and `openOutputStream` run in `viewModelScope` on Main before any dispatcher switch. A provider that fetches the file during open blocks the UI thread.

How it happens:

On a new phone, pick a backup from a cloud provider over a slow connection.

What the user experiences:

A frozen screen with no spinner drawn, on the main new-phone journey.

Impact:

A possible ANR during restore.

Root cause:

The supplier is invoked outside `withContext(Dispatchers.IO)`.

Evidence:

app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:153-156

```
153  viewModelScope.launch {
154      _uiState.update { it.copy(isValidating = true, validationError = null, previewData = null) }
155      try {
156          val stream = inputStreamSupplier()
```

Assumptions:

A cloud provider blocks during open. This could not be verified here, which is why this is Potential.

Recommended fix:

Call the suppliers inside `withContext(Dispatchers.IO)`.

Regression test:

A test supplier that records its thread, asserted not to be the main thread, for both export and validate.

Related findings:

None

BUG-051 P3 A failed figures flow stays dead for the whole process, unlike the rank

Confidence Potential

Category Reliability: silent failure

Area Cached figures

Location app/src/main/java/com/macrobase/app/data/cache/FiguresCache.kt:78-92 (figures)
app/src/main/java/com/macrobase/app/data/repository/rank/RankRepositoryImpl.kt:124-131
app/src/main/java/com/macrobase/app/core/di/DiModules.kt:128, 150

What is wrong:

If `source()` throws inside `shareIn`, the sharing coroutine ends, the handler only logs, and the dead SharedFlow stays in `live`. Later `figures()` calls for that key return the same dead flow. The rank's protection (`retryWhen`) was not applied to this sibling.

How it happens:

Any exception from the Room flow or calculation, such as an SQLiteException from disk or low-memory conditions. Afterwards that range or month never updates until the process dies.

What the user experiences:

Calendar: an empty grid with "0 days" and "0%", and no error. Weight, Consistency and Averages share one `combine`, so one dead key before its first value leaves them all on a spinner.

Impact:

A screen that stays wrong until the app is killed.

Root cause:

The rank's "never stops for good" protection was not applied to the sibling cache.

Evidence:

app/src/main/java/com/macrobase/app/data/cache/FiguresCache.kt:82-88

```
82  val shared = flow {
83      store?.read(key, codec)?.let { emit(it.value) }
84      emitAll(source().onEach { value -> store?.write(key, "", value, codec) })
85  }
86      .distinctUntilChanged()
87      .shareIn(CoroutineScope(scope.coroutineContext + job), watching, replay = 1)
88  live[key] = Live(shared, job, watching)
```

app/src/main/java/com/macrobase/app/data/repository/rank/RankRepositoryImpl.kt:124-130

```
124  // A replay that fails is logged and tried again a little later, so the shared
125  // replay never stops for good and never takes the app down
126  .retryWhen { cause, attempt ->
127      Log.w(TAG, "Rank replay failed, trying again: ${cause.message}")
128      delay(RETRY_DELAY_MS * (attempt + 1))
129      true
130  }
```

Assumptions:

The kotlinx `shareIn` failure behaviour is known from its documentation, not executed here. No common trigger was found.

Recommended fix:

Add the same `retryWhen` with backoff to the upstream of `figures`, or remove the key from `live` on failure.

Regression test:

A source that throws once then emits: assert a subscriber receives the value after virtual-time backoff.

Related findings:

None

BUG-052 P3 On a fresh install, a second save the same day makes the pending strategy active today

Confidence Confirmed

Category Bug: next-day rule

Area Goals

Location app/src/main/java/com/macrobase/app/data/repository/GoalsRepositoryImpl.kt:259-281 (updateGoals)
docs/RANKING_SYSTEM.md:32-34

What is wrong:
When a save asks for the pending strategy, `strategyChanged` is false, so the else branch runs. If `FITNESS_GOAL` or `MAINTENANCE_CALORIES` was never stored, that branch writes the requested values, which are the pending ones, as the active ones.

How it happens:

1. Fresh install, with both keys null.
2. Save 1 picks Cutting at maintenance 2,500. It goes to SCHEDULED_*, and the active keys stay null.
3. Save 2 the same day, for example a macro tweak, sends the scheduled values, so `asksForPending` is true.
4. The else branch stores FITNESS_GOAL = CUTTING and MAINTENANCE = 2500.
5. The next read rewrites today's history record to Cutting.

What the user experiences:
Today becomes Cutting, where it should stay Maintaining until tomorrow. Cancel would no longer restore the old strategy.

Impact:
Low harm, mainly on day one. It breaks the rule in `RANKING_SYSTEM.md`.

Root cause:
"Fill in the missing keys" uses the requested goal rather than the current one.

Evidence:

app/src/main/java/com/macrobase/app/data/repository/GoalsRepositoryImpl.kt:263-265

```
263  val asksForPending = pendingFitnessGoal != null &&
264      goals.fitnessGoal == pendingFitnessGoal && goals.maintenanceCalories == pendingMaintenance
265  val strategyChanged = !asksForActive && !asksForPending
```

app/src/main/java/com/macrobase/app/data/repository/GoalsRepositoryImpl.kt:275-280

```
275  if (prefs[Keys.FITNESS_GOAL] == null) {
276      prefs[Keys.FITNESS_GOAL] = goals.fitnessGoal.name
277  }
278  if (prefs[Keys.MAINTENANCE_CALORIES] == null) {
279      prefs[Keys.MAINTENANCE_CALORIES] = goals.maintenanceCalories
280  }
```

Assumptions:
None.

Recommended fix:
In that branch store `currentFitnessGoal` and `currentMaintenance`, or fill the keys unconditionally at the top of `updateGoals`.

Regression test:
Fresh store, `updateGoals(goal(CUTTING, 2500))` twice: assert today is MAINTAINING, the change is still pending, and Cutting applies tomorrow.

Related findings:
BUG-007

BUG-053 P3 The catalog's sha256 pins break on a Windows checkout

Confidence Highly Likely
Category Build-tool safety
Area Catalog tooling
Location tools/catalog/build_catalog.py:389, 453-473 (check_sources)
tools/catalog/sources.csv
.gitattributes (not present)

What is wrong:
The pins hash the raw bytes of files git treats as text. `core.autocrlf=true` is set system-wide on this machine and there is no `.gitattributes`. The files are LF now, but a fresh clone or a checkout that rewrites them here produces CRLF.

How it happens:
Clone the repository on Windows with the default Git config, then run `build_catalog.py`.

What the user experiences:
`build_catalog.py`, `--check` and `python -m unittest` all fail, with "changed since it was imported".

Impact:
It fails closed, so no bad data ships, but the message sends the maintainer to re-pin a hash for an unchanged file. `tools/catalog/` is not committed yet, so the first commit sets this up.

Root cause:
Hashing line-ending-sensitive bytes without pinning line endings in git.

Evidence:

tools/catalog/build_catalog.py:389

```
389  def file_sha256(path):
```

tools/catalog/build_catalog.py:469-472

```
469      problems.append(f"sources.csv: {imported} does not exist")
470  elif file_sha256(path) != s.get("imported_sha256", ""):
471      problems.append(f"sources.csv: {imported} changed since it was imported. Put corrections in
overrides.csv; "
472                      f"after a real re-import, set imported_sha256 to {file_sha256(path)}")
```

git config --show-origin core.autocrlf

```
file:E:/softwares/git/Git/etc/gitconfig true
```

Assumptions:
Git's standard conversion on checkout. No fresh clone was made.

Recommended fix:
Add `.gitattributes` with `eol=lf` for `*.csv` and `*.fingerprint`, or hash with line endings normalised.

Regression test:
A unit test that `file_sha256` of a CRLF copy still passes, or a repository check that the attributes exist.

Related findings:
BUG-065

P4: Informational (14)

BUG-054 [P4] The Profile "Time zone" setting has no effect, and its UTC default looks real

Confidence Confirmed

Category UX: dead setting

Area Preferences

Location app/src/main/java/com/macrobase/app/feature/preferences/PreferencesScreen.kt:715-722
app/src/main/java/com/macrobase/app/data/repository/PreferencesRepositoryImpl.kt:62
app/src/main/java/com/macrobase/app/core/util/DeviceClock.kt:13

What is wrong:
The field accepts any text, shows "UTC" before anything is set, and is only stored and copied into backups. Every date in the app uses the device time zone.

How it happens:
Type "America/New_York" (or nonsense) and Save. It reports success, and nothing changes.

What the user experiences:
A setting that looks like it controls where the day starts, and does not.

Impact:
Misleading. It also adds a useless text field to the privacy data map.

Root cause:
A stored setting with no consumer and no validation.

Evidence:

app/src/main/java/com/macrobase/app/feature/preferences/PreferencesScreen.kt:715-722

```
715  MbTextField(
716      value = form.timeZone,
717      onValueChange = viewModel::onTimeZoneChanged,
718      label = "Time zone",
719      helpText = "For example UTC or America/New_York",
720      keyboardOptions = KeyboardOptions(imeAction = ImeAction.Next),
721      modifier = Modifier.fillMaxWidth()
722  )
```

app/src/main/java/com/macrobase/app/data/repository/PreferencesRepositoryImpl.kt:62

```
62  timeZone = prefs[Keys.TIME_ZONE] ?: "UTC",
```

app/src/main/java/com/macrobase/app/core/util/DeviceClock.kt:13

```
13  override fun getZone(): ZoneId = ZoneId.systemDefault()
```

Assumptions:
None.

Recommended fix:
Remove the field. Do not apply it to date logic without a separate design, because that would re-date existing entries.

Regression test:
If removed, assert old backups still import.

Related findings:
None

BUG-055 P4 The calendar treats today differently from the heatmap and the BUG-032 rule

Confidence Confirmed

Category UX: inconsistency

Area Calendar

Location app/src/main/java/com/macrobase/app/feature/calendar/CalendarScreen.kt:231-239 (calculateMonthState)
app/src/main/java/com/macrobase/app/feature/calendar/CalendarScreen.kt:463-469 (calendarCellColors)

What is wrong:

An unlogged today is drawn in the Missed colour, while the heatmap draws it as an outline. A half-eaten today also joins `loggedDays` as Under, so "% days of green" dips each morning.

How it happens:

10 of 10 green days is 100%. After breakfast it is 10 of 11, or 91%. By evening it is 11 of 11.

What the user experiences:

A false Missed colour every morning, and a percentage that falls and recovers.

Impact:

Cosmetic. The percentage is documented behaviour, but the colour contradicts the code comment that today is not a missed day.

Root cause:

`calendarCellColors` only special-cases future days.

Evidence:

app/src/main/java/com/macrobase/app/feature/calendar/CalendarScreen.kt:231-234

```
231  val eligiblePastOrTodayDays = currentMonthCells.filter { it.date.isBefore(today) || (it.isToday &&
it.loggedCalories > 0.0) }
232
233  val daysMissed = eligiblePastOrTodayDays.count { it.loggedCalories <= 0.0 }
234  val loggedDays = eligiblePastOrTodayDays.filter { it.loggedCalories > 0.0 }
```

app/src/main/java/com/macrobase/app/feature/calendar/CalendarScreen.kt:463-469

```
463  internal fun calendarCellColors(cell: CalendarDayCellModel): Pair<Color, Color> = when {
464      !cell.isCurrentMonth -> Color.Transparent to AppColors.TextMuted.copy(alpha = 0.35f)
465      cell.isFuture && cell.performanceCategory == CalendarPerformanceConfig.PerformanceCategory.EMPTY_MISSED -
>
466          Color.Transparent to AppColors.TextMuted
467      else -> CalendarPerformanceConfig.getColorForCategory(cell.performanceCategory) to
468          CalendarPerformanceConfig.getTextColorForCategory(cell.performanceCategory)
469  }
```

Assumptions:

None.

Recommended fix:

Treat an unlogged today like a future cell, and leave today out of the green percentage until it ends.

Regression test:

Assert `calendarCellColors` is transparent for an unlogged today.

Related findings:

BUG-029

BUG-056 P4 A weight logged on a future day vanishes from the list and graph

Confidence Confirmed
Category Validation
Area Weight
Location app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:117-124, 251-253, 451, 466-470

What is wrong:
Next Day has no upper limit, and logging accepts any date. Every preset range ends today, and delete exists only on list rows, so a stray future entry is hard to remove.

How it happens:
Tap Next Day once by accident, log a weight, then look at the list. The count does not change. Deleting it requires a Custom range with a future end date.

What the user experiences:
The new entry does not appear.

Impact:
Minor. Future days are intentional for food, but a future body weight has no such purpose.

Root cause:
Preset ranges end at today, but entry dates are not limited to today.

Evidence:

app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:119

```
119  WeightTimeInterval.LAST_30_DAYS -> windowStart(today, 30) to today
```

app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:123

```
123  WeightTimeInterval.ALL_TIME -> LocalDate.of(2000, 1, 1) to today
```

app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:251-252

```
251  fun onNextDay() {
252      _selectedDate.value = _selectedDate.value.plusDays(1)
```

Assumptions:
Treated as unintended for weight only.

Recommended fix:
Stop weight logging past today, or include entries beyond today in All Time.

Regression test:
Assert a future weight is refused, or that it appears in the All Time list.

Related findings:
None

BUG-057 [P4] Any weight edit silently erases that entry's note

Confidence Confirmed

Category Data integrity

Area Weight

Location app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:543, 601
app/src/main/java/com/macrobase/app/domain/usecase/BiometricAndGoalUseCases.kt:51-54
app/src/main/java/com/macrobase/app/data/database/dao/UserDaos.kt:184-185

What is wrong:
Log and Edit both create a new entry with `note = null` and `id = 0`. The unique date index plus REPLACE deletes the old row, so the note goes with it, and the id and `createdAt` change too.

How it happens:
An entry with a note exists, the user edits its weight, and the note is gone. Saving the dialog unchanged also erases it.

What the user experiences:
Nothing visible, since no screen shows notes.

Impact:
Notes can only arrive from imported backups, which is why this is P4.

Root cause:
Edit is implemented as a fresh insert, not an update of the existing entry.

Evidence:

app/src/main/java/com/macrobase/app/feature/weight/WeightScreen.kt:601

```
601  viewModel.logWeight(entry.date, kg)
```

app/src/main/java/com/macrobase/app/domain/usecase/BiometricAndGoalUseCases.kt:53

```
53  return weightRepository.addWeightEntry(WeightEntry(date = date, weightKg = weightKg, note = note))
```

app/src/main/java/com/macrobase/app/data/database/dao/UserDaos.kt:184-185

```
184  @Insert(onConflict = OnConflictStrategy.REPLACE)
185  suspend fun insertWeightEntry(entry: WeightEntryEntity): Long
```

Assumptions:
Some users hold notes from imported backups.

Recommended fix:
Carry the existing note through on edit, or update the existing entry.

Regression test:
Against real Room: insert with a note, edit through the use case, and assert the note stays with one row for that date.

Related findings:
None

BUG-058 [P4] The Overwrite warning omits goals and profile, and mentions a "50k food catalog"

Confidence Confirmed

Category UX: inaccurate warning

Area Backup and restore

Location app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:459-464
app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:455, 470-472

What is wrong:

The warning lists diary entries, custom foods, recipes, weight and water, but not goals and profile, which Overwrite also replaces. It also says "Your built-in 50k food catalog", left over from an older catalog: the shipped catalogs hold 977 and 1,632 foods. The failure note says "goals and profile" even when only one failed.

How it happens:

Any Overwrite restore.

What the user experiences:

A destructive dialog that understates what it replaces.

Impact:

Users can lose a goal setup they did not expect to lose.

Root cause:

UI copy was not updated with BUG-016 and later changes.

Evidence:

app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:459

```
459  text = "This action will completely REPLACE all your existing food diary entries, custom foods, recipes, weight history, and water logs with the backup data.",
```

app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:464

```
464  text = "Note: Your built-in 50k food catalog will NOT be affected.",
```

built_in_foods.db (read-only query): foods in built_in_foods.db

```
977
```

branded_foods.db (read-only query): foods in branded_foods.db

```
1632
```

Assumptions:

None.

Recommended fix:

Name goals and profile in the dialog, remove the catalog size, and report which of the two settings failed.

Regression test:

A unit test for the failure note.

Related findings:

BUG-067

BUG-059 P4 The user's own recipes are not searchable, though the README says they are

Confidence Confirmed

Category UX: expectation gap

Area Search

Location app/src/main/java/com/macrobase/app/data/repository/FoodRepositoryImpl.kt:51-59 (searchFoods)
README.md:31

What is wrong:

`searchFoods` queries custom foods and the catalogs only. The code comment says recipes are logged from the Recipes screen, so this looks deliberate, but the README lists recipes among what search covers, and the empty state gives no pointer.

How it happens:

Save a recipe, then search its name on Home.

What the user experiences:

"No Foods Found".

Impact:

A dead end for a user who expects search to find everything they own.

Root cause:

Food Detail cannot open a recipe id, so search leaves recipes out.

Evidence:

app/src/main/java/com/macrobase/app/data/repository/FoodRepositoryImpl.kt:51-59

```
51  override suspend fun searchFoods(query: String, limit: Int): List<Food> {
52      val customMatches = customFoodDao.searchCustomFoods(query, limit).map { it.toDomainFood() }
53      val remainingLimit = limit - customMatches.size
54      val builtInMatches = if (remainingLimit > 0) {
55          localDatabaseProvider.searchFoods(query, remainingLimit)
56      } else emptyList()
57
58      return customMatches + builtInMatches
59  }
```

README.md:31

```
31  - **Fast Offline Search**: Instant food search over the built-in Indian food catalog, packaged products sold
in India (Open Food Facts), custom foods and recipes: word starts in any order, other spellings and Hindi names,
everyday foods first.
```

Assumptions:

Excluding recipes is a deliberate design choice. The README is then what is wrong.

Recommended fix:

List matching recipes in search and open the recipe log sheet from there, or correct the README and add a pointer to the Recipes view in the empty state.

Regression test:

A search test that a saved recipe appears, or a UI test for the pointer.

Related findings:

BUG-067

BUG-060 P4 Search queries are written to logcat in release builds

Confidence Confirmed

Category Privacy: logging

Area Search

Location app/src/main/java/com/macrobase/app/data/provider/LocalFoodDatabaseProvider.kt:69
docs/SECURITY_PRIVACY_AUDIT.md:29
app/proguard-rules.pro

What is wrong:
`Log.d` prints each query. R8 does not strip it, because `proguard-rules.pro` has no `-assumenosideeffects` rule for `android.util.Log`. `SECURITY_PRIVACY_AUDIT.md` says production builds emit zero user data in logs.

How it happens:
Type any food search in a release build.

What the user experiences:
Nothing visible on the device.

Impact:
Very low risk: other apps cannot read logcat without a privileged permission, but it reaches adb and shared bug reports.

Root cause:
A debug log line left unguarded.

Evidence:

app/src/main/java/com/macrobase/app/data/provider/LocalFoodDatabaseProvider.kt:69

```
69  android.util.Log.d("LocalFoodDb", "Query '$trimmed' matched ${foods.size} foods")
```

docs/SECURITY_PRIVACY_AUDIT.md:29

```
29  | **Sensitive Log Leakage** | Production Logcat leaking user weight or food logs | Production builds emit zero user data in logs; technical logs restricted to catalog asset initialization | **MITIGATED (Low)** |
```

Assumptions:
None.

Recommended fix:
Remove the query text from the log line, or guard it with `BuildConfig.DEBUG`.

Regression test:
Extend `ProjectRulesLintTest` to reject interpolated user text in log calls.

Related findings:
None

BUG-061 P4 Restore's name fallback can relink an entry to a different food

Confidence Confirmed

Category Data integrity: id remapping (BUG-006 sibling)

Area Backup and restore

Location app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:556-565 (foodIdResolver)
app/src/main/java/com/macrobase/app/data/repository/DiaryFoodLinkRepair.kt:42-43

What is wrong:

`DiaryFoodLinkRepair` refuses to adopt a different food created later with the old name, using a `createdAt` check. The restore resolver has no such check.

How it happens:

1. Food A "Protein Shake" is logged, then renamed.
2. New food B is created as "Protein Shake".
3. Export writes a null uuid for the old entries, because the name no longer matches.
4. Restore's unique-name match picks B.

What the user experiences:

Before the restore these entries opened from their own snapshot. After it they open as B: null secondary nutrients fill from B, and editing the quantity recalculates from B (BUG-008).

Impact:

Rare. It needs a rename, a new food with the old name, a restore and then an edit. Logged calories stay until the entry is edited.

Root cause:

Two link-repair paths with different rules.

Evidence:

app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:557-558

```
557  val rowId = dto.customFoodUuid?.let { foodRowIdByUuid[it] }
558      ?: foodsByName[name]?.singleOrNull()?.id
```

app/src/main/java/com/macrobase/app/data/repository/DiaryFoodLinkRepair.kt:42-43

```
42  val match = foodsByName[normalize(entry.foodName)]?.singleOrNull() ?: continue
43  if (match.createdAt > entry.createdAt) continue
```

Assumptions:

None.

Recommended fix:

Apply the same `createdAt` check in the fallback. A failed check unlinks, which is the safe snapshot path.

Regression test:

Build that sequence, export, Overwrite, and assert the entry is unlinked and opens from its snapshot.

Related findings:

BUG-008

BUG-062 [P4] Imported values are not range-checked, and a far-out date makes every later export fail

Confidence Confirmed

Category Input validation

Area Backup and restore

Location app/src/main/java/com/macrobase/app/data/portability/BackupJsonSerializer.kt:142-153, 317-319, 347-349
app/src/main/java/com/macrobase/app/data/portability/BackupArchiveManager.kt:247-254
app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:75, 138, 148

What is wrong:

Dates, negatives and absurd values pass straight to the database. A missing required field silently becomes a default (date 1970, 0 kcal, `gramWeight` 100). A manifest with no `checksums`, or blank hashes, skips the checksum step, though counts and the ZIP CRC still apply.

How it happens:

A hand-edited or shared file with counts intact and blank hashes, for example `dateEpochDay` beyond LocalDate's range.

What the user experiences:

The bad entry appears on no screen, so it cannot be deleted. Every later export fails with "Invalid value for EpochDay" until a good Overwrite. Negative calories show as negative totals.

Impact:

It requires a deliberately edited or hostile file. Normal use cannot produce these rows.

Root cause:

Validation covers archive structure, not field values.

Evidence:

app/src/main/java/com/macrobase/app/data/portability/BackupJsonSerializer.kt:142

```
142  dateEpochDay = obj.getLong("dateEpochDay") ?: 0L,
```

app/src/main/java/com/macrobase/app/data/portability/BackupArchiveManager.kt:251

```
251  if (expectedHash.isNotBlank() && !actualHash.equals(expectedHash, ignoreCase = true)) {
```

app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:75

```
75  dateString = LocalDate.ofEpochDay(e.dateEpochDay).toString(),
```

Assumptions:

A deliberately edited or hostile file.

Recommended fix:

Validate records before Overwrite (date range, finite non-negative numbers, required fields), and require a checksum for every file the format defines.

Regression test:

Backups with an out-of-range date, a negative value, a missing required field and a blank hash must each be refused.

Related findings:

None

BUG-063 [P4] Export is not a consistent snapshot, and stays available during a restore

Confidence Confirmed

Category Concurrency

Area Backup and restore

Location app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:48-54
app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:336-348

What is wrong:
Export reads five tables in separate statements with no transaction, and the Export button depends only on `isExporting`.

How it happens:
Tap Export while a restore is running. If the commit lands between the reads, the diary is pre-restore and the foods post-restore. Row-id lookups then fail, so uuids are dropped.

What the user experiences:
A backup whose counts and checksums pass, but whose content is mixed.

Impact:
Effectively limited to the user's own double action. No other background writer exists except the startup link repair.

Root cause:
No read transaction and no mutual exclusion.

Evidence:

app/src/main/java/com/macrobase/app/data/repository/PortabilityRepositoryImpl.kt:49-50

```
49  val diaryEntities = userDatabase.diaryDao().getAllEntries()
50  val customFoodEntities = userDatabase.customFoodDao().getAllCustomFoods()
```

app/src/main/java/com/macrobase/app/feature/importexport/ImportExportScreen.kt:336

```
336  if (uiState.isExporting) {
```

Assumptions:
None.

Recommended fix:
Wrap the reads in a transaction, and disable Export while restoring.

Regression test:
Assert the Export button is disabled in the `isImporting` state.

Related findings:
BUG-010

BUG-064 P4 Export does not check import's limits, so one oversized record makes a whole backup unrestorable

Confidence Confirmed

Category Export and import asymmetry

Area Backup and restore

Location app/src/main/java/com/macrobase/app/data/portability/BackupArchiveManager.kt:51, 71-124
app/src/main/java/com/macrobase/app/data/portability/BackupJsonSerializer.kt:590-592

What is wrong:

Records over 16,384 JSON characters fail parsing. Export reports success without checking this or re-validating its own output, and one bad record refuses the entire restore.

How it happens:

A custom food name pasted at about 16K characters. The form has no length limit, and the name is copied into each diary entry that logs the food.

What the user experiences:

"Corrupted custom_foods.json: Record at position N is longer than 16384 characters".

Impact:

It needs an absurd name, but the result is a backup that cannot be restored.

Root cause:

The limits live only on the import side.

Evidence:

app/src/main/java/com/macrobase/app/data/portability/BackupArchiveManager.kt:51

```
51  const val MAX_RECORD_CHARS = 16 * 1024L
```

app/src/main/java/com/macrobase/app/data/portability/BackupJsonSerializer.kt:590-592

```
590  if (position >= recordEnd) {
591      throw IllegalArgumentException("Record at position $recordStart is longer than ${recordEnd - recordStart}
characters")
592  }
```

Assumptions:

Needs an absurdly long name, hence P4.

Recommended fix:

Cap name fields, and run the validator over the archive before reporting success.

Regression test:

Export a food with a 20K name, and assert either a refused input or a backup that validates.

Related findings:

BUG-049

BUG-065 P4 A deleted catalog version row cannot be re-pinned while the content is unchanged

Confidence Confirmed

Category Build-tool safety

Area Catalog tooling

Location tools/catalog/build_catalog.py:1294-1318
tools/catalog/build_branded.py:102-123

What is wrong:

The pin sits only in the write branch. README step 3 says to delete the row and rebuild. If content is unchanged, `current and bundled` is true, so the build says "nothing written" and pins nothing. The same trap follows a crash between the fingerprint write and the pin.

How it happens:

Delete the row for the current version from `catalog_versions.csv`, then run `build_catalog.py`.

What the user experiences:

A maintainer loop with no explanation. The only way out is deleting the database or fingerprint, or adding the row by hand.

Impact:

Maintainer time only. It fails closed.

Root cause:

Pinning is tied to writing the database.

Evidence:

tools/catalog/build_catalog.py:1305-1306

```
1305  elif current and bundled:
1306      status = "The committed catalog is already current; nothing written."
```

tools/catalog/build_catalog.py:1315-1318

```
1315  if version_problems is None:
1316      with open(VERSIONS_CSV, "a", encoding="utf-8", newline="") as f:
1317          csv.writer(f, lineterminator="\n").writerow((DATABASE_VERSION, fingerprint,
date.today().isoformat()))
1318      status += f"\nPinned catalog version {DATABASE_VERSION} in catalog_versions.csv"
```

Assumptions:

None.

Recommended fix:

Pin whenever `version_problems is None`, outside the write branch, in both scripts.

Regression test:

Run `main` with a versions file holding no row and a current database, and assert the row is added.

Related findings:

BUG-053

BUG-066 P4 A variable shadowing bug turns a recipe error into a traceback

Confidence Confirmed
Category Build-tool error handling
Area Catalog tooling
Location tools/catalog/build_catalog.py:774, 802, 865

What is wrong:
`known` starts as a set of recipe ids, then is rebound to a bool in the nutrient loop.

How it happens:
After one good recipe, a later ingredient with a mistyped `MacroBase:` id, or one naming a recipe further down, hits `in known` on a bool and raises TypeError.

What the user experiences:
A traceback rather than the normal problem list. The exit code is still non-zero.

Impact:
Maintainer convenience only.

Root cause:
The same name is used for two meanings.

Evidence:

tools/catalog/build_catalog.py:774

```
774  known = {r.get("id", "") for r in recipes}
```

tools/catalog/build_catalog.py:802

```
802  later = ingredient.startswith("MacroBase:") and ingredient[len("MacroBase:"):] in known
```

tools/catalog/build_catalog.py:865

```
865  known = totals[column] is not None and unstated[column] <= UNSTATED_SHARE * dry
```

Assumptions:
None.

Recommended fix:
Rename the inner variable.

Regression test:
A forward reference placed after a recipe that builds, expecting the normal "a recipe below this one" problem.

Related findings:
None

BUG-067 [P4] Documentation disagrees with the code in several places

Confidence Confirmed

Category Maintainability: documentation drift

Area Documentation

Location README.md:51
docs/KNOWN_PITFALLS_AND_GUARDRAILS.md:23
docs/SECURITY_PRIVACY_AUDIT.md:74
docs/PRIVACY_DATA_MAP.md:17, 33
docs/BASKET_ARCHITECTURE.md:11, 25
docs/RANKING_SYSTEM.md:162
docs/CALCULATION_ENGINE.md:170
docs/BACKUP_FORMAT.md:216

What is wrong:

- README and the pitfalls doc give old database versions (3 and 4). The code is 5.
- The security audit says "Broadcast Receivers: None". The widget provider is an exported receiver.
- The privacy map says the user can "clear all data". No delete-all action exists.
- The privacy map and security audit describe a 7,966-food USDA database. The shipped catalogs are smaller and INDB-based.
- The basket doc says the basket is in memory and cleared on process death. It is persisted since BUG-015.
- The rank doc says 30 history days to a page. The code and test use 15.
- The calculation doc makes today always eligible. The code counts it only once logged (BUG-032).
- The backup doc says Merge updates goals and preferences. The code keeps the phone's.

How it happens:
Reading the documents.

What the user experiences:
Nothing directly. The privacy map's "clear all data" is the one claim a user could rely on.

Impact:
It misleads later fixes, especially as `AGENTS.md` asks agents to treat these documents as guardrails.

Root cause:
Documents were not updated alongside later fixes.

Evidence:

README.md:51

```
51  - `macrobase_user.db`: Room database (v3) storing user diary entries (with historical snapshot macros and secondary nutrients: fiber, sugar, sodium), custom foods, recipes, weight, and water logs.
```

docs/KNOWN_PITFALLS_AND_GUARDRAILS.md:23

```
23  - Database schema is at `USER_DATABASE_VERSION = 4`.
```

docs/SECURITY_PRIVACY_AUDIT.md:74

```
74  - Broadcast Receivers: None.
```

docs/PRIVACY_DATA_MAP.md:33

```
33  - **Scope**: Contains only user-created data. Built-in 7,966-food USDA database is **never** exported into the backup archive.
```

docs/BASKET_ARCHITECTURE.md:11

```
11  - **Cleared**: On explicit submission (successful transaction commit to `DiaryRepository`), explicit cancellation, or application process death. It does not survive application restart, ensuring stale partial logs do not persist indefinitely.
```

docs/RANKING_SYSTEM.md:162

```
162  - The history lists every counted day, 30 to a page: the first page is today and the 29 days
```

app/src/main/java/com/macrobase/app/feature/rank/RankScreen.kt:461

```
461  private const val HISTORY_PAGE_DAYS = 15
```

docs/BACKUP_FORMAT.md:216

```
216  - **Goals & Preferences**: Updated to backup configuration.
```

Assumptions:

None.

Recommended fix:

Update these documents, and add a data-erasure action or remove the "clear all data" claim.

Regression test:

Not needed for the documents.

Related findings:

BUG-058, BUG-059

F. User journey failures

Each journey in this codebase, where it can fail, and the findings responsible.

Journey	Where it can fail	Findings
First run and onboarding	Goal setup can schedule a strategy that then applies a day early. A phone in Light mode shows near-invisible status icons.	BUG-052, BUG-021
Finding a food	Plurals and short phrases return nothing. Typing can flash an error card. A food created from the empty state still shows as not found. Saved recipes cannot be found. Branded products can outrank plain foods. Queries reach logcat.	BUG-004, BUG-024, BUG-025, BUG-059, BUG-046, BUG-060
Logging food to a day (main workflow)	The search pill ignores the day on Home and defaults to Breakfast. A double tap can double a copied meal. Some catalog values are far off. Macro ring numbers clip at large font sizes.	BUG-003, BUG-023, BUG-009, BUG-042, BUG-043, BUG-045, BUG-044, BUG-030
Editing or deleting history	Changing an amount re-bases the entry on the food's current values. Past macro rings use today's split. One tap deletes weigh-ins and water logs.	BUG-008, BUG-020, BUG-005
Creating custom foods and recipes	Rotation or a trip to the scanner wipes the form. Recipe servings silently become 1. Grams typed for an ingredient multiply by 100. Ingredient search can be stale or crash. A failed save crashes the app.	BUG-002, BUG-018, BUG-019, BUG-026, BUG-022, BUG-005
Scanning a nutrition label	Wrong values with a green tick on shared lines, claim lines, spaced headers, inverted columns and unit changes. A clear table can report nothing found. Modals, camera failures and a denied permission lead to dead ends.	BUG-001, BUG-012, BUG-013, BUG-014, BUG-015, BUG-016, BUG-017, BUG-031, BUG-032, BUG-033, BUG-034, BUG-035, BUG-041, BUG-036, BUG-037, BUG-038, BUG-039, BUG-040
Water and weight tracking	The screens stay on yesterday after midnight, and rotation resets the chosen day. "Edit Today" misleads. Future entries vanish, and an edit drops the note.	BUG-006, BUG-027, BUG-028, BUG-056, BUG-057, BUG-022
Goals, rank and statistics	A weight edit can replace the strategy. Merge drops goal history. The calendar's today goes stale or reads as missed. A failed figures flow stays dead.	BUG-007, BUG-011, BUG-029, BUG-055, BUG-051
Backup, restore and moving phones	Back during restore leaves it half applied. Back during export leaves a 0-byte file. Merge duplicates edited water logs. Very large exports crash. Cloud files can freeze the screen.	BUG-010, BUG-011, BUG-048, BUG-047, BUG-049, BUG-050, BUG-058, BUG-061, BUG-062, BUG-063, BUG-064
Settings and profile	The time zone field does nothing. Docs promise a clear-all action that does not exist.	BUG-054, BUG-067
Maintainers and the catalog build	A Windows clone breaks the pins. A deleted version row cannot be re-pinned. A curator error can raise a traceback. Tests cover one catalog.	BUG-053, BUG-065, BUG-066, BUG-046

G. Root-cause patterns

Recurring causes behind several findings. Fixing a pattern is cheaper than fixing its findings one at a time.

Numbers are tied to rows, not to the words beside them

The parser finds a row by keyword, then picks a number by position. Nearly every wrong-value scanner bug follows from this, along with header words being tested one at a time.

Findings: BUG-001, BUG-013, BUG-015, BUG-014, BUG-016, BUG-032

Validation that exists but is not on the live path

Unit conversion, the serving cross-check and OCR pass comparison are all implemented and tested in isolation, but the real path does not call them. Tests pass while the feature does nothing, and the green tick claims a check that did not run.

Findings: BUG-017, BUG-034, BUG-041

Test fixtures kinder than real input

Scanner fixtures use multi-word OCR elements, while ML Kit reports one word per element. Search tests use one catalog, while the app merges two. Room tests inline the transaction executor, so cancellation timing cannot show up.

Findings: BUG-015, BUG-014, BUG-046, BUG-010

A fix applied to one screen, not its siblings

BUG-040 (midnight), BUG-011 (double tap), BUG-047 (memory), BUG-049 (goal history) and BUG-006 (food links) each fixed the reported path. Sibling paths were left out. Before a fix is closed, search for the same shape elsewhere.

Findings: BUG-006, BUG-027, BUG-029, BUG-023, BUG-049, BUG-020, BUG-061

Must-finish work tied to a screen's lifetime

Restore and export run in `viewModelScope`, so Back cancels them halfway. A multi-store operation needs an owner that outlives the screen.

Findings: BUG-010, BUG-048

"The day" is captured once and scattered

Dates are read at creation and defaulted separately at each entry point. There is no single source for today or the day being logged.

Findings: BUG-003, BUG-006, BUG-027, BUG-029

Form state lives in composition

`remember` holds work the user would be upset to lose. With no `configChanges`, any configuration change discards it.

Findings: BUG-002, BUG-027

No shared failure path for writes

Food Detail, Basket and Preferences report failures well. Other screens launch fire-and-forget writes that crash on error, and a broad `catch (e: Exception)` also swallows cancellation in two places.

Findings: BUG-022, BUG-026, BUG-024, BUG-010

History modelled as a current setting

Goals are per-day history, but Merge and the strategy flow treat them as one setting. Macro splits also reach past days from the live goal.

Findings: BUG-011, BUG-007, BUG-020

Plausibility checks keyed on names, not on density

Catalog checks rely on keyword lists and one-size caps, so per-serving rows and unit errors slip through whenever the name misses a list.

Findings: BUG-009, BUG-042, BUG-043, BUG-045, BUG-044

Limits and docs that live on one side

Import has limits that export ignores, and documents have not kept pace with fixes.

Findings: BUG-064, BUG-049, BUG-067, BUG-058

H. Recommended fix order

Ordered by value for effort. Early steps are small and remove the most risk, and step 3 is the largest single piece of work.

1. Stop the misleading signal (hours)

Remove or qualify the scanner's green ticks until the parser is fixed. Add a BackHandler that blocks leaving during restore and export. Fix the status bar style, the "Edit Today" label and the Overwrite warning text. These are one-line changes that remove real risk at once.

Findings: BUG-041, BUG-010, BUG-048, BUG-021, BUG-028, BUG-058

2. Protect history and restores

Move restore and export to an application-lifetime scope, merge goal history in Merge, and rescale from the snapshot in Edit Mode. Stop weight edits replacing a chosen strategy.

Findings: BUG-010, BUG-048, BUG-011, BUG-008, BUG-007

3. Rework the parser's binding of numbers to labels

First add a word-level OCR fixture builder, so tests stop hiding the problem. Then bind numbers to keywords, join "100" and "g" in headers, restrict anchors to the header row, assign cells by X range, and start at the table header. This one piece of work closes most of the scanner group.

Findings: BUG-001, BUG-013, BUG-012, BUG-014, BUG-015, BUG-016, BUG-017, BUG-031, BUG-032

4. Make the day and the forms survive

One logging-context helper, the BUG-040 pattern on the trackers and calendar, the applied-once guard, and saveable form state.

Findings: BUG-003, BUG-006, BUG-027, BUG-029, BUG-002

5. Search and logging quality

Plural handling, a cancellation-safe search, undo for deletes, the one-shot "+" sheet, recipe validation and a portion picker.

Findings: BUG-004, BUG-024, BUG-025, BUG-005, BUG-023, BUG-018, BUG-019, BUG-026

6. Failure paths

A shared write-failure pattern across ViewModels, memory handling on export, IO-thread stream opening, and a retry in FiguresCache.

Findings: BUG-022, BUG-049, BUG-050, BUG-051

7. Catalog data and tools

Density-based checks in the importer and build, then corrections for the listed foods. Add `.gitattributes` before the first commit of `tools/catalog`.

Findings: BUG-053, BUG-009, BUG-042, BUG-043, BUG-044, BUG-045, BUG-046, BUG-065, BUG-066

8. Remaining low-priority items and documents

Work through the remaining P3 and P4 items as those areas are next touched, and refresh the documents.

Findings: BUG-020, BUG-052, BUG-047, BUG-033, BUG-034, BUG-035, BUG-036, BUG-037, BUG-038, BUG-039, BUG-040, BUG-030, BUG-054, BUG-055, BUG-056, BUG-057, BUG-059, BUG-060, BUG-061, BUG-062, BUG-063, BUG-064, BUG-067

I. Test plan

The most important tests to add, in priority order. Each finding also names its own regression test.

Test	What it does	Covers
Word-level OCR fixture builder	Build scanner fixtures from label text with one element per word and realistic geometry, then replay the existing golden labels through it.	BUG-015, BUG-014
Real-layout parser corpus	Add side-by-side, paragraph-style, FSSAI per-100/per-serve/%RDA, claim-line, ingredients-first, spaced header and mixed-unit samples, all through `parse()`, asserting values and not just presence.	BUG-001, BUG-013, BUG-012, BUG-017
Restore cancellation test	Use real Room with the real transaction executor, cancel mid-transaction, and assert the database and settings moved together.	BUG-010
Merge goal history test	Merge a backup into a phone with its own later history, then assert per-day goals, the rank start day and the current goal.	BUG-011
Snapshot immutability test	Log, change the food, edit the amount, and assert the per-unit basis came from the snapshot.	BUG-008
Strategy stability test	Weight edits with a blank target and with a set target, on a strategy already in force.	BUG-007
Day rollover suite	With a mutable clock, advance past midnight for Water, Weight, Calendar and Stats, covering a month boundary, and assert writes go to the new day while a picked day stays.	BUG-006, BUG-029
Logging context test	For each entry point on Home, on today and on a past day, assert the route's date and meal.	BUG-003
State restoration tests	StateRestorationTester on the custom food form, recipe form and trackers, and navigate-and-return through the scanner.	BUG-002, BUG-027
First Compose UI tests	The repository has none. Start with double tap on the "+" sheet, scanner modal touch blocking, delete undo, and font scale on the macro ring.	BUG-023, BUG-037, BUG-005, BUG-030
Write failure tests	A throwing repository for each ViewModel, asserting an error is shown, nothing escapes, and a retry works.	BUG-022, BUG-026
Search through both catalogs	Build `CatalogSearchTests` with both managers, and add plural and phrase cases.	BUG-046, BUG-004
Catalog plausibility checks	Energy density and water by difference for cooked dishes, sodium ceilings by category, and all-zero products.	BUG-009, BUG-042, BUG-043, BUG-045
Device checks that static review cannot replace	On a real phone: keyboard over forms with edge-to-edge on, Light mode status bar, font scale 2.0, TalkBack through the scanner, a restore from a cloud provider, and real label photos.	BUG-021, BUG-050

J. Remaining uncertainty

Nothing was executed. The app was not built, run or tested, so no finding has been reproduced on a device, and the state of the existing tests is unknown. "Confirmed" means confirmed by reading and tracing the code.

- **Real ML Kit output.** Scanner findings rest on word-level elements and on the geometry assumed in each sample. BUG-014, BUG-015 and BUG-016 depend on this most.
- **How common each label format is.** Scanner priorities are judgement calls based on Indian, US, EU and Australian layouts. No sample of real labels was available.
- **Tilted photos.** There is no deskew, and rows are grouped by absolute Y. By estimate, about 2 degrees can orphan values and about 4 degrees can shift them a row. This could not be tested and is not a finding.
- **Keyboard behaviour.** The app has edge-to-edge on, with no `imePadding` or `windowSoftInputMode`. Forms and dialogs may have fields or Save buttons under the keyboard. This needs a device check and is not a finding.
- **Compose and library runtime behaviour.** Sheet interactivity during hide (BUG-023), hit-testing (BUG-037), `enableEdgeToEdge` defaults (BUG-021) and `shareIn` failure handling (BUG-051) come from knowledge of those libraries, not from running them. Room's transaction behaviour (BUG-010) was read from its bytecode.
- **Timing.** The windows in BUG-024, BUG-010 and BUG-048 were not measured.
- **Memory limits.** The export threshold in BUG-049 depends on device heap and diary size.
- **Cloud storage providers.** Blocking during open (BUG-050) is unverified. Whether a provider reports a late upload failure after `close()` is also unknown. If it does not, "exported successfully" could be premature, since export never re-reads the file.
- **Source data.** The INDB workbook, USDA and CoFID downloads and the Open Food Facts snapshot are not in the repository, so source fidelity rests on the pinned hashes, and the size of the errors in BUG-009 and BUG-044 is inferred. Whether INDB's lack of a stated licence is acceptable is a legal question outside this audit.
- **Device transfer.** Android's real device-to-device behaviour under the backup rules was read, not tried.
- **Licences.** Completeness of third-party notices for ML Kit and other dependencies was not verified.
- **Dependencies.** Versions could not be checked online for known vulnerabilities, and no version was recognised as seriously vulnerable.
- **Working tree.** The audit covers files as they stood, including uncommitted catalog work. Some files named here, such as `tools/catalog/` and `branded_foods.db`, are not yet committed.